arXiv:2502.16982v1 [cs.LG] 24 Feb 2025

MUON IS SCALABLE FOR LLM TRAINING

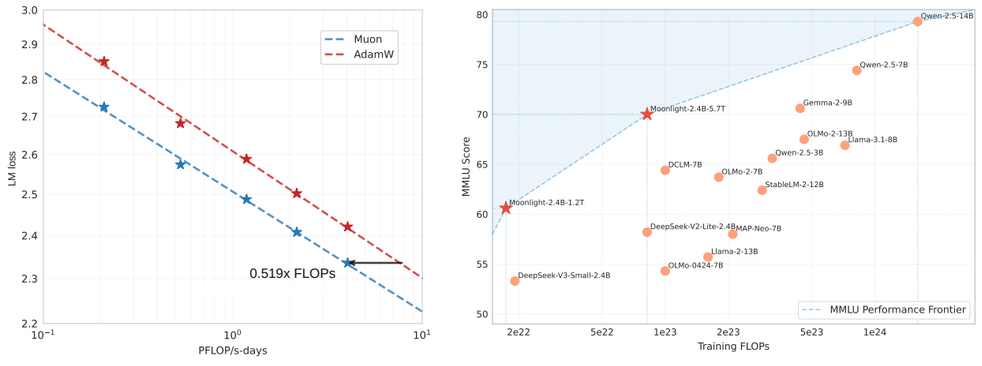
TECHNICAL REPORT

Jingyuan Liu[1] **Jianlin Su**[1] **Xingcheng Yao**[2] **Zhejun Jiang**[1] **Guokun Lai**[1] **Yulun Du**[1] **Yidao Qin**[1] **Weixin Xu**[1] **Enzhe Lu**[1] **Junjie Yan**[1] **Yanru Chen**[1] **Huabin Zheng**[1] **Yibo Liu**[1] **Shaowei Liu**[1] **Bohong Yin**[1] **Weiran He**[1] **Han Zhu**[1] **Yuzhi Wang**[1] **Jianzhou Wang**[1] **Mengnan Dong**[1] **Zheng Zhang**[1] **Yongsheng Kang**[1] **Hao Zhang**[1] **Xinran Xu**[1] **Yutao Zhang**[1] **Yuxin Wu**[1] **Xinyu Zhou**[1] * **Zhilin Yang**[1]

[1] Moonshot AI [2] UCLA

ABSTRACT

Recently, the Muon optimizer (K. Jordan et al. 2024) based on matrix orthogonalization has demonstrated strong results in training small-scale language models, but the scalability to larger models has not been proven. We identify two crucial techniques for scaling up Muon: (1) adding weight decay and (2) carefully adjusting the per-parameter update scale. These techniques allow Muon to work out-of-the-box on large-scale training without the need of hyper-parameter tuning. Scaling law experiments indicate that Muon achieves $\sim 2\times$ computational efficiency compared to AdamW with compute optimal training. Based on these improvements, we introduce Moonlight, a 3B/16B-parameter Mixture-of-Expert (MoE) model trained with 5.7T tokens using Muon. Our model improves the current Pareto frontier, achieving better performance with much fewer training FLOPs compared to prior models. We open-source our distributed Muon implementation that is memory optimal and communication efficient. We also release the pretrained, instruction-tuned, and intermediate checkpoints to support future research.



(a) (b)

Figure 1: Scaling up with Muon. **(a)** Scaling law experiments comparing Muon and Adam. Muon is $\sim 2\times$ more computational efficient than Adam with compute optimal training. **(b)** The MMLU performance of our Moonlight model optimized with Muon and other comparable models. Moonlight advances the Pareto frontier of performance vs training FLOPs.

*Corresponding author: zhouxinyu@moonshot.cn

arXiv:2502.16982v1 [cs.LG] 24 Feb 2025

Muon 可扩展用于大语言模型训练

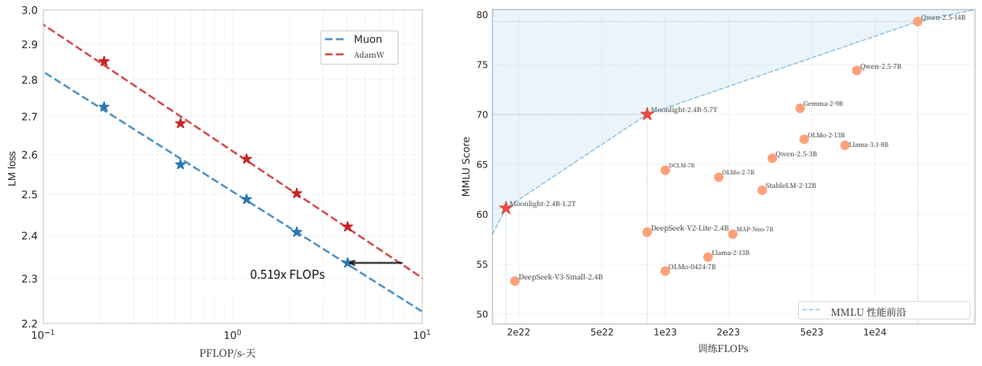
技术报告

Jingyuan Liu[1] **Jianlin Su**[1] **Xingcheng Yao**[2] **Zhejun Jiang**[1] **Guokun Lai**[1] **Yulun Du**[1] **Yidao Qin**[1] **Weixin Xu**[1] **Enzhe Lu**[1] **Junjie Yan**[1] **Yanru Chen**[1] **Huabin Zheng**[1] **Yibo Liu**[1] **Shaowei Liu**[1] **Bohong Yin**[1] **Weiran He**[1] **Han Zhu**[1] **Yuzhi Wang**[1] **Jianzhou Wang**[1] **Mengnan Dong**[1] **Zheng Zhang**[1] **Yongsheng Kang**[1] **Hao Zhang**[1] **Xinran Xu**[1] **Yutao Zhang**[1] **Yuxin Wu**[1] **Xinyu Zhou**[1] * **Zhilin Yang**[1]

[1] 月之跃人工智能 [2] 加州大学洛杉矶分校

摘要

最近，基于矩阵正交化的Muon优化器（K. Jordan等人2024年）在训练小规模语言模型方面表现出色，但其扩展到更大规模模型的性能尚未得到验证。我们识别出两种提升Muon扩展性的关键技术：(1) 添加权重衰减和(2) 精心调整每参数更新尺度。这些技术使Muon无需超参数调优即可直接应用于大规模训练。缩放律实验表明，与采用计算最优训练的AdamW相比，Muon实现了 $\sim 2\times$ 计算效率。基于这些改进，我们推出了Moonlight，这是一个使用Muon训练的3B/16B参数专家混合（MoE）模型，训练时使用了5.7T个token。我们的模型改进了当前的Pareto前沿，相较于先前模型，在显著减少训练FLOPs的同时实现了更好的性能。我们开源了内存优化且通信高效的分布式Muon实现，并发布了预训练、指令微调和中间检查点，以支持未来研究。



(a) (b)

图1：使用Muon进行扩展。 **(a)** 比较Muon和Adam的扩展规律实验。Muon在计算最优训练中比Adam更高效。 **(b)** 我们的Moonlight模型使用Muon和其他可比模型优化的MMLU性能。Moonlight将性能与训练FLOPs的Pareto前沿推进。

*通讯作者：zhouxinyu@moonshot.c n

1 Introduction

The rapid advancement of large language models (LLMs) (OpenAI et al. 2024; DeepSeek-AI et al. 2024; Grattafiori et al. 2024; Gemini Team et al. 2024) has significantly pushed forward the progress in artificial general intelligence. However, training capable LLMs remains a computationally intensive and resource-demanding process due to scaling laws (Kaplan et al. 2020; Hoffmann et al. 2022). Optimizers play a crucial role in efficiently and effectively training of LLMs, with Adam (Kingma et al. 2015) and its variant AdamW (Loshchilov et al. 2019) being the standard choice for most large-scale training.

Recent developments in optimization algorithms have shown potential to improve training efficiency beyond AdamW (Liu et al. 2024; K. Jordan et al. 2024; Yuan et al. 2024; Vyas et al. 2025; X.-L. Li 2018a; X.-L. Li 2018b; Pooladzandi et al. 2024; X. Li 2022; X.-L. Li 2024; Pethick et al. 2025). Among these, K. Jordan et al. 2024 proposed Muon, which updates matrix parameters with orthogonalized gradient momentum using Newton-Schulz iteration. Initial experiments with Muon have demonstrated promising results in small-scale language model training. However, as discussed in this blog (K. Jordan et al. 2024), several critical challenges remain unaddressed: (1) how to effectively scale optimizers based on matrix orthogonalization to larger models with billions of parameters trained with trillions of tokens, (2) how to compute approximate orthogonalization in a distributed setting, and (3) whether such optimizers can generalize across different training stages including pre-training and supervised finetuning (SFT).

In this technical report, we present a comprehensive study addressing these challenges. Our work builds upon Muon while systematically identifying and resolving its limitations in large-scale training scenarios. Our technical contributions include:

- **Analysis for Effective Scaling of Muon**: Through extensive analysis, we identify that weight decay plays a crucial role in Muon's scalability. Besides, we propose scale adjustments to Muon's parameter-wise update rule. Such adjustments allow Muon to work out-of-the-box without hyper-parameter tuning, and also significantly improve training stability.
- **Efficient Distributed Implementation**: We develop a distributed version of Muon with ZeRO-1 (Rajbhandari et al. 2020) style optimization, achieving optimal memory efficiency and reduced communication overhead while preserving the mathematical properties of the algorithm.
- **Scaling Law Validation**: We performed scaling law research that compares Muon with strong AdamW baselines, and showed the superior performance of Muon (1a). Based on the scaling law results, Muon achieves comparable performance to AdamW trained counterparts while requiring only approximately 52% of the training FLOPs.

Our comprehensive experiments demonstrate that Muon can effectively replace AdamW as the de facto optimizer for large-scale LLM training, offering significant improvements in both training efficiency and model performance. As a result of this work, we release Moonlight, a 16B-parameter MoE model trained using Muon, along with our implementation and intermediate training checkpoints to facilitate further research in scalable optimization techniques for LLMs.

2 Methods

2.1 Background

The Muon Optimizer Muon (K. Jordan et al. 2024) has recently been proposed to optimize neural network weights representable as matrices. At iteration t, given current weight $\mathbf{W}_{t-1}$, momentum μ, learning rate η_t and objective $\mathcal{L}_t$, the update rule of the Muon optimizer can be stated as follows:

$$\begin{aligned}\mathbf{M}_t &= \mu\mathbf{M}_{t-1} + \nabla\mathcal{L}_t(\mathbf{W}_{t-1})\\ \mathbf{O}_t &= \text{Newton-Schulz}(\mathbf{M}_t)^{[1]}\\ \mathbf{W}_t &= \mathbf{W}_{t-1} - \eta_t\mathbf{O}_t\end{aligned} \tag{1}$$

Here, $\mathbf{M}_t$ is the momentum of gradient at iteration t, set as a zero matrix when $t = 0$. In Equation 1, a Newton-Schulz iteration process (Bernstein et al. 2024) is adopted to approximately solve $(\mathbf{M}_t\mathbf{M}_t^{\mathrm{T}})^{-1/2}\mathbf{M}_t$. Let $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathrm{T}} = \mathbf{M}_t$ be the singular value decomposition (SVD) of $\mathbf{M}_t$, we will have $(\mathbf{M}_t\mathbf{M}_t^{\mathrm{T}})^{-1/2}\mathbf{M}_t = \mathbf{U}\mathbf{V}^{\mathbf{T}}$, which orthogonalizes $\mathbf{M}_t$. Intuitively, orthogonalization can ensure that the update matrices are isomorphic, preventing the weight from learning along a few dominant directions (K. Jordan et al. 2024).

[1] In practice, we follow (K. Jordan et al. 2024) to use a Nesterov-style momentum by putting $\mu\mathbf{M}_t + \nabla\mathcal{L}_t(\mathbf{W}_{t-1})$ to the Newton-Schulz iteration instead of $\mathbf{M}_t$.

1 引言

大型语言模型（LLM）（OpenAI等人2024年；DeepSeek-AI等人2024年；Grattafiori等人2024年；Gemini Team等人2024年）的快速发展显著推动了通用人工智能的进展。然而，由于缩放律（Kaplan等人2020年；Hoffmann等人2022年）的影响，训练具备能力的LLM仍然是一个计算密集且资源需求巨大的过程。优化器在高效且有效地训练LLM方面发挥着关键作用，Adam（Kingma等人2015年）及其变体AdamW（Loshchilov等人2019年）是大多数大规模训练的标准选择。

优化算法的最新发展显示出超越 AdamW 的潜力，能够提高训练效率（Liu 等人 2024；K. Jordan 等人 2024；Yuan 等人 2024；Vyas 等人 2025；X.-L. Li 2018a；X.-L. Li 2018b；Pooladzandi 等人 2024；X. Li 2022；X.-L. Li 2024；Pethick 等人 2025）。在这些方法中，K. Jordan 等人 2024 提出了 Muon，它使用牛顿-舒尔茨迭代对正交化的梯度动量进行矩阵参数更新。初步实验表明，Muon 在小规模语言模型训练中取得了有前景的结果。然而，正如本博客所讨论的（K. Jordan 等人 2024），仍然存在几个关键挑战尚未解决：(1) 如何有效地将基于矩阵正交化的优化器扩展到具有数十亿参数、使用万亿个 token 进行训练的大模型上；(2) 如何在分布式环境中计算近似正交化；(3) 这种优化器是否能在包括预训练和监督微调（SFT）在内的不同训练阶段泛化。

在本次技术报告中，我们提出了一项全面的研究，旨在解决这些挑战。我们的工作基于Muon，同时系统地识别并解决了其在大规模训练场景中的局限性。我们的技术贡献包括：

- **Muon有效扩展性分析**：通过深入分析，我们确定权重衰减在Muon的可扩展性中起着关键作用。此外，我们提出了对Muon参数级更新规则的缩放调整。此类调整使Muon能够开箱即用而无需超参数调整，并显著提高了训练稳定性。
- **高效分布式实现**：我们开发了具有ZeRO-1（Rajbhandari等人，2020年）风格优化的Muon分布式版本，在保持算法数学特性的同时，实现了最优内存效率和降低通信开销。
- **缩放律验证**：我们进行了缩放律研究，将Muon与强AdamW基线进行了比较，并展示了Muon的优越性能（1a）。基于缩放律结果，Muon在仅需要约52%的训练FLOPs的情况下，实现了与AdamW训练对应项相当的性能。

我们的综合实验表明，Muon 可以有效地替代 AdamW，成为大规模大语言模型训练的实用优化器，并在训练效率和模型性能方面带来显著改进。作为这项工作的成果，我们发布了 Moonlight，这是一个使用 Muon 训练的 16B 参数 MoE 模型，以及我们的实现和中间训练检查点，以促进可扩展优化技术在 LLM 方面的进一步研究。

2 方法

2.1 背景

Muon 优化器 Muon (K. Jordan 等人 2024) 最近被提出用于优化可以表示为矩阵的神经网络权重。在迭代 t 时，给定当前权重 $\mathbf{W}_{t-1}$、动量 μ、学习率 η_t 和目标 $\mathcal{L}_t$，Muon 优化器的更新规则可以表述如下：

$$\begin{aligned}\mathbf{M}_t &= \mu\mathbf{M}_{t-1} + \nabla\mathcal{L}_t(\mathbf{W}_{t-1})\\ \mathbf{O}_t &= \text{Newton-Schulz}(\mathbf{M}_t)^{[1]}\\ \mathbf{W}_t &= \mathbf{W}_{t-1} - \eta_t\mathbf{O}_t\end{aligned} \tag{1}$$

这里，$\mathbf{M}_t$ 是迭代 t处的梯度动量，当 $t = 0$时将其设为零矩阵。在公式1中，采用牛顿-舒尔茨迭代过程（Bernstein等人，2024）来近似求解$(\mathbf{M}_t\mathbf{M}_t^{\mathrm{T}})^{-1/2}\mathbf{M}_t$。令$\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathrm{T}} = \mathbf{M}_t$是$\mathbf{M}_t$的奇异值分解（SVD），我们将得到$(\mathbf{M}_t\mathbf{M}_t^{\mathrm{T}})^{-1/2}\mathbf{M}_t = \mathbf{U}\mathbf{V}^{\mathbf{T}}$，这可以正交化$\mathbf{M}_t$。直观上，正交化可以确保更新矩阵是同构的，防止权重沿着少数几个主导方向学习（K. Jordan等人，2024）。

[1] 在实践中，我们遵循 (K. Jordan 等人 2024) 使用 Nesterov 风格动量，通过将 $\mu\mathbf{M}_t + \nabla\mathcal{L}_t(\mathbf{W}_{t-1})$ 放入牛顿-舒尔茨迭代，而不是 $\mathbf{M}_t$。

Newton-Schulz Iterations for Matrix Orthogonalization Equation 1 is calculated in an iterative process. At the beginning, we set $\mathbf{X}_0 = \mathbf{M}_t / \|\mathbf{M}_t\|_{\mathrm{F}}$. Then, at each iteration k, we update $\mathbf{X}_k$ from $\mathbf{X}_{k-1}$ as follows:

$$\mathbf{X}_k = a\mathbf{X}_{k-1} + b(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^{\mathrm{T}})\mathbf{X}_{k-1} + c(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^{\mathrm{T}})^2\mathbf{X}_{k-1} \tag{2}$$

where $\mathbf{X}_N$ is the result of such process after N iteration steps. Here a, b, c are coefficients. In order to ensure the correct convergence of Equation 2, we need to tune the coefficients so that the polynomial $f(x) = ax + bx^3 + cx^5$ has a fixed point near 1. In the original design of K. Jordan et al. 2024, the coefficients are set to $a = 3.4445$, $b = -4.7750$, $c = 2.0315$ in order to make the iterative process converge faster for small initial singular values. In this work, we follow the same setting of coefficients.

Steepest Descent Under Norm Constraints Bernstein et al. 2024 proposed to view the optimization process in deep learning as steepest descent under norm constraints. From this perspective, we can view the difference between Muon and Adam (Kingma et al. 2015; Loshchilov et al. 2019) as the difference in norm constraints. Whereas Adam is a steepest descent under the a norm constraint dynamically adjusted from a Max-of-Max norm, Muon offers a norm constraint that lies in a static range of Schatten-p norm for some large p (Franz 2024). When equation 1 is accurately computed, the norm constraint offered by Muon will be the spectral norm. Weights of neural networks are used as operators on the input space or the hidden space, which are usually (locally) Euclidean (Cesista 2024), so the norm constraint on weights should be an induced operator norm (or spectral norm for weight matrices). In this sense, the norm constraint offered by Muon is more reasonable than that offered by Adam.

2.2 Scaling Up Muon

Weight Decay While Muon performs significantly better than AdamW on a small scale as shown by K. Jordan et al. 2024, we found the performance gains diminish when we scale up to train a larger model with more tokens. We observed that both the weight and the layer output's RMS keep growing to a large scale, exceeding the high-precision range of bf16, which might hurt the model's performance. To resolve this issue, we introduced the standard AdamW (Loshchilov et al. 2019) weight decay mechanism into Muon[2].

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(\mathbf{O}_t + \lambda\mathbf{W}_{t-1}) \tag{3}$$

We experimented on Muon both with and without weight decay to understand its impact on the training dynamics of LLMs. Based on our scaling law research in Sec 3.2, we trained an 800M parameters model with 100B tokens ($\sim 5\times$ optimal training tokens). Figure 2 shows validation loss curves of the model trained with AdamW, vanilla Muon (without weight decay), and Muon with weight decay. While vanilla Muon initially converges faster, we observed that some model weights grew too large over time, potentially limiting the model's long-term performances. Adding weight decay addressed this issue - the results demonstrate that Muon with weight decay outperforms both vanilla Muon and AdamW, achieving lower validation loss in the over-train regime. Therefore, we adjusted our update rule to equation 3, where λ is the weight decay ratio.

Consistent update RMS An important property of Adam and AdamW (Kingma et al. 2015, Loshchilov et al. 2019) is that they maintain a theoretical update RMS around 1[3]. However, we show that Muon's update RMS varies depending on the shape of the parameters, according to the following lemma:

Lemma 1. *For a full-rank matrix parameter of shape* $[A, B]$*, its theoretical Muon update RMS is* $\sqrt{1/\max(A, B)}$.

The proof can be found in the Appendix A. We monitored Muon's update RMS during training and found it typically close to the theoretical value given above. We note that such inconsistency can be problematic when scaling up the model size:

- When $\max(A, B)$ is too large, e.g. the dense MLP matrix, the updates become too small, thus limiting the model's representational capacity and leading to suboptimal performances;
- When $\max(A, B)$ is too small, e.g. treating each KV head in GQA (Shazeer 2019) or MLA (DeepSeek-AI et al. 2024) as a separate parameter, the updates become too large, thus causing training instabilities and leading to suboptimal performances as well.

[2]The original implementation of Muon omits weight decay. A recent concurrent work in Muon incorporates weight decay and demonstrates improved performance. See this commit and this discussion.

[3]Due to Adam's $\beta_1 < \beta_2$ and $\epsilon > 0$, the actual update RMS is usually less than 1.

牛顿-舒尔茨迭代用于矩阵正交化公式1是在迭代过程中计算的。最初，我们设置$\mathbf{X}_0 = \mathbf{M}_t / \|\mathbf{M}_t\|_{\mathrm{F}}$。然后，在每次迭代 k 时，我们从$\mathbf{X}_{k-1}$ 更新$\mathbf{X}_k$，如下所示：

$$\mathbf{X}_k = a\mathbf{X}_{k-1} + b(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^{\mathrm{T}})\mathbf{X}_{k-1} + c(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^{\mathrm{T}})^2\mathbf{X}_{k-1} \tag{2}$$

其中 $\mathbf{X}_N$ 是经过 N 次迭代步骤后的结果。这里 a, b, c 是系数。为了确保方程 2 的正确收敛，我们需要调整系数，使得多项式 $f(x) = ax + bx^3 + cx^5$ 在 1 附近有一个不动点。在 K. Jordan 等人 2024 年的原始设计中，系数被设置为 $a = 3.4445$, $b = -4.7750$,$c = 2.0315$，以使迭代过程对于小的初始奇异值收敛得更快。在这项工作中，我们遵循相同的系数设置。

范数约束下的最速下降法 Bernstein 等人 2024 年提出将深度学习中的优化过程视为范数约束下的最速下降法。从这个角度来看，我们可以将 Muon 和 Adam (Kingma 等人 2015 年；Loshchilov 等人 2019 年) 之间的差异视为范数约束的差异。Adam 是在动态调整的 a 范数约束下的最速下降法，而 Muon 提供的是一个静态的 Schatten-p 范数范围内的范数约束，适用于某些较大的 p (Franz 2024)。当方程 1 被准确计算时，Muon 提供的范数约束将是谱范数。神经网络的权重被用作输入空间或隐藏空间上的算子，这些空间通常是（局部）欧几里得的 (Cesista 2024)，因此权重的范数约束应该是诱导算子范数（或权重矩阵的谱范数）。从这个意义上说，Muon 提供的范数约束比 Adam 更合理。

2.2 扩展 Muon

权重衰减 虽然 K. Jordan 等人 2024 年的研究表明 Muon 在小规模上显著优于 AdamW，但我们在扩展到使用更多训练令牌训练更大模型时发现性能提升会减弱。我们观察到权重和层输出的均方根（RMS）会持续增长到很大规模，超出 bf16 的高精度范围，这可能会损害模型性能。为了解决这个问题，我们将标准 AdamW（Loshchilov 等人 2019 年）的权重衰减机制引入 Muon[2]。

我们通过带有和不带权重衰减的实验来研究其对大语言模型（LLM）训练动态的影响。基于我们在第 3.2 节中的扩展定律研究，我们使用 $\sim 5\times$ optimal training tokens 训练了一个 800M 参数的模型。图 2 展示了使用 AdamW、不带权重衰减的原始 Muon 以及带权重衰减的 Muon 训练的模型验证损失曲线。虽然原始 Muon 初始收敛速度更快，但我们观察到一些模型权重随时间增长过大，可能限制模型的长期性能。添加权重衰减解决了这个问题——结果表明，带权重衰减的 Muon 性能优于原始 Muon 和 AdamW，在过训练阶段实现了更低的验证损失。因此，我们将更新规则调整为公式 3，其中 λ 是权重衰减率。

一致的更新均方根Adam 和 AdamW（Kingma 等人，2015 年；Loshchilov 等人，2019 年）的一个重要特性是它们在 1^3 周围保持一个理论上的更新均方根。然而，我们证明 Muon 的更新均方根根据以下引理随参数的形状而变化：

引理 1。对于 *full-rank* 矩阵参数，形状为 $[A, B]$，其理论 *Muon* 更新均方根 *RMS* 为 $\sqrt{1/\max(A, B)}$.

证明可在附录A中找到。我们监控了训练期间Muon的更新均方根，发现它通常接近上述理论值。我们注意到，这种不一致性在扩展模型大小时可能会出现问题：

- 当$\max(A, B)$过大时，例如稠密的MLP矩阵，更新值变得过小，从而限制了模型的表征能力并导致性能欠佳；
- 当 $\max(A, B)$ 太小时，例如将 GQA（Shazeer 2019）或 MLA（DeepSeek-AI 等人 2024）中的每个 KV 头视为独立参数，更新量会变得过大，从而引起训练不稳定，并导致性能不佳。

[2]Muon 的原始实现忽略了权重衰减。最近的一项 Muon 并行工作集成了权重衰减并展示了性能提升。参见此提交和此讨论。[3]由于 Adam 的 $\beta_1 < \beta_2$ 和 $\epsilon > 0$，实际更新均方根通常小于 1。

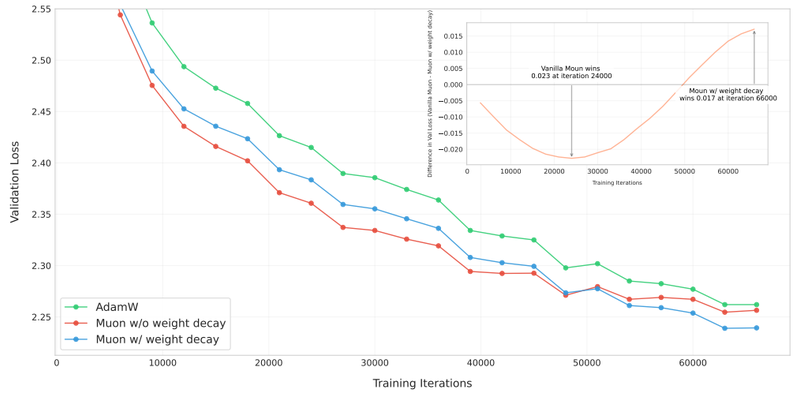


Figure 2: Validation loss curves for AdamW (green), Muon without weight decay (red), and Muon with weight decay (blue).

In order to maintain consistent update RMS among matrices of different shapes, we propose to scale the Muon update for each matrix by its $\sqrt{\max(A, B)}$ to cancel the effect of Lemma 1 [4] . Experiments in Sec 3.1 show that this strategy is beneficial for optimization.

Matching update RMS of AdamW Muon is designed to update matrix-based parameters. In practice, AdamW is used in couple with Muon to handle non-matrix based parameters, like RMSNorm, LM head, and embedding parameters. We would like the optimizer hyper-parameters (learning rate η, weight decay λ) to be shared among matrix and non-matrix parameters.

We propose to match Muon's update RMS to be similar to that of AdamW. From empirical observations, AdamW's update RMS is usually around 0.2 to 0.4. Therefore, we scale Muon's update RMS to this range by the following adjustment:

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t \cdot \sqrt{\max(A, B)} + \lambda \mathbf{W}_{t-1}) \tag{4}$$

We validated this choice with empirical results (see Appendix A for details). Moreover, we highlighted that with this adjustment, Muon can directly **reuse** the learning rate and weight decay tuned for AdamW.

Other Hyper-parameters Muon contains two other tunnable hyper-parameters: Newton-Schulz iteration steps and momentum μ. We empirically observe that when setting N to 10, the iterative process will yield a more accurate orthogonalization result than $N = 5$, but it won't lead to better performances. Hence we set $N = 5$ in this work for the sake of efficiency. We do not see a consistent performance gain in tuning momentum, so we chose 0.95, same as K. Jordan et al. 2024.

2.3 Distributed Muon

ZeRO-1 and Megatron-LM Rajbhandari et al. 2020 introduced the ZeRO-1 technique that partitions the expensive optimizer states (e.g. master weights, momentum) all over the cluster. Megatron-LM (Shoeybi et al. 2020) integrated ZeRO-1 into its native parallel designs. Based on Megatron-LM's sophisticated parallel strategies, e.g. Tensor-Parallel (TP), Pipeline Parallel (PP), Expert Parallel (EP) and Data Parallel (DP), the communication workload of ZeRO-1 can be reduced from gathering all over the distributed world to only gathering over the data parallel group.

Method ZeRO-1 is efficient for AdamW because it calculates updates in an element-wise fashion. However, Muon requires the full gradient matrix to calculate the updates. Therefore, vanilla ZeRO-1 is not directly applicable to Muon.

[4] K. Jordan et al. 2024's original implementation scales the updates by $\sqrt{\max(1, A/B)}$, which is equivalent to our proposal (up to a global scale) if all matrices have the same second dimension; Pethick et al. 2025 and You 2025 discussed a similar issue on update scaling factors concurrently to our work.

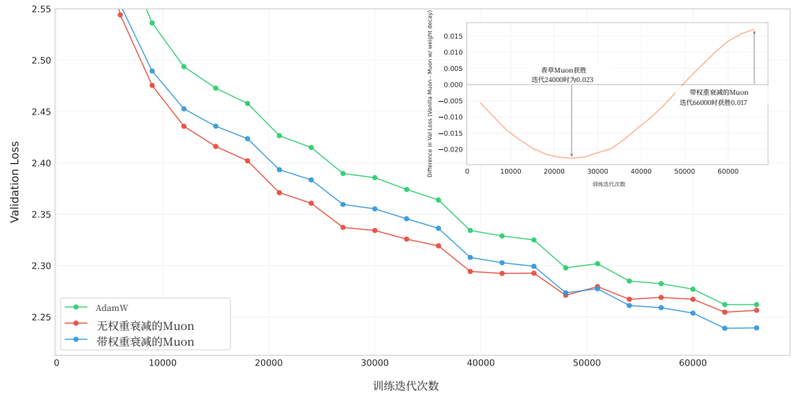


图2：验证 在 AdamW （绿色）的损失曲线上，没有权重衰减的Muon（红色），以及带有权重衰减的Muon（蓝色）).

为了在形状不同的矩阵之间保持一致的更新均方根，我们提出通过每个矩阵的 $\sqrt{\max(A, B)}$ 对 Muon 更新进行缩放，以消除引理 1 [4] 的影响。第 3.1 节的实验表明，该策略有利于优化。

匹配AdamW的更新RMSMuon旨在更新基于矩阵的参数。在实践中，AdamW通常与Muon配合使用，以处理非基于矩阵的参数，例如RMSNorm、LM头和嵌入参数。我们希望优化器超参数（学习率 η、权重衰减 λ）能够被矩阵和非矩阵参数共享。

我们建议将Muon的更新均方根匹配为与AdamW相似。根据经验观察，AdamW的更新均方根通常在0.2到0.4之间。因此，我们通过以下调整将Muon的更新均方根缩放到该范围：

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t \cdot \sqrt{\max(A, B)} + \lambda \mathbf{W}_{t-1}) \tag{4}$$

我们通过实证结果验证了这一选择（详情请参见附录A）。此外，我们强调，通过这种调整，Muon可以直接**重用** 为AdamW调整的学习率和权重衰减。

其他超参数 Muon 包含两个其他可调节的超参数：Newton-Schulz迭代步骤和动量 μ。我们通过经验观察到，当将 N 设置为10时，迭代过程将产生比 $N = 5$更精确的正交化结果，但它不会带来更好的性能。因此，为了效率，我们在本文中设置了 $N = 5$ 。我们没有在调节动量时看到一致的性能提升，因此选择了0.95，与K. Jordan等人2024年的选择相同。

2.3 分布式Muon

ZeRO-1 和 Megatron-LM Rajbhandari 等人 2020 年引入了 ZeRO-1 技术，该技术将昂贵的优化器状态（例如主权重、动量）在整个集群中进行划分。Megatron-LM (Shoeybi 等人 2020 年) 将 ZeRO-1 集成了其本地的并行设计中。基于 Megatron-LM 的复杂并行策略，例如张量并行（TP）、流水线并行（PP）、专家并行（EP）和数据并行（DP），ZeRO-1 的通信工作负载可以从在整个分布式世界中收集减少到仅在数据并行组中收集。

方法 ZeRO-1 对 AdamW 很高效，因为它以逐元素的方式计算更新。然而，Muon 需要完整的梯度矩阵来计算更新。因此，纯 ZeRO-1 不能直接应用于 Muon。

[4] K. Jordan 等人 2024 年的原版实现通过 $\sqrt{\max(1, A/B)}$ 对更新进行缩放，这等效于我们的提议（在全局缩放方面），如果所有矩阵具有相同的第二维度；Pethick 等人 2025 年和 You 2025 年同时讨论了我们工作中的更新缩放因子上的类似问题。

Algorithm 1 Distributed Muon

Require: Full Gradients $\mathbf{G}$, DP partitioned Momentum $\mathbf{m}$, DP partitioned parameters $\mathbf{p}$, momentum μ.

// Reduce-scatter G on DP for correct gradients
$\mathbf{g}$ = reduce_scatter($\mathbf{G}$, dp_group)
// Apply momentum to $\mathbf{g}$ using local partitioned momentum $\mathbf{m}$
$\mathbf{g}'$ = update_with_momentum($\mathbf{g}, \mathbf{m}, \mu$)
// DP Gather: gathering $\mathbf{g}'$ across DP into a full matrix $\mathbf{G}$
$\mathbf{G}$ = gather($\mathbf{g}'$, dp_group)
// Calculate Muon update
$\mathbf{U}$ = Newton-Schulz($\mathbf{G}$)
// Discard the rest of $\mathbf{U}$ and only keep the local partition $\mathbf{u}$, then apply the update rule
$\mathbf{p}'$ = apply_update($\mathbf{p}, \mathbf{u}$)
// All-gather updated $\mathbf{p}'$ into $\mathbf{P}$
$\mathbf{P}$ = all_gather($\mathbf{p}'$, dp_group)
// Return the update RMS for logging
return $\sqrt{\mathbf{u}^2\texttt{.mean}()}$

We propose a new distributed solution based on ZeRO-1 for Muon, referred to as Distributed Muon. Distributed Muon follows ZeRO-1 to partition the optimizer states on DP, and introduces two additional operations compared to a vanilla Zero-1 AdamW optimizer:

1. `DP Gather.` For a local DP partitioned master weight ($1/DP$ the size of the model weight), this operation is to gather the corresponding partitioned gradients into a full gradient matrix.
2. `Calculate Full Update.` After the above gathering, perform Newton-Schulz iteration steps on the full gradient matrix as described in Sec 2.1. Note that we will then discard part of the full update matrix, as we only need the partition corresponding to the local parameters to perform update.

The implementation of Distributed Muon is described in Algorithm 1. The additional operations introduced by Distributed Muon are colored in blue.

Analysis We compared Distributed Muon to a classic ZeRO-1 based distributed AdamW (referred as Distributed AdamW for simplicity) in several aspects:

- `Memory Usage.` Muon uses only one momentum buffer, while AdamW uses two momentum buffers. Therefore, the additional memory used by the Muon optimizer is half of Distributed AdamW.
- `Communication Overhead.` For each device, the additional DP gathering is only required by the local DP partitioned parameters $\mathbf{p}$. Therefore, the communication cost is less than the reduce-scatter of $\mathbf{G}$ or the all-gather of $\mathbf{P}$. Besides, Muon only requires the Newton-Schulz iteration steps in bf16, thus further reducing the communication overhead to 50% comparing to fp32. Overall, the communication workload of Distributed Muon is $(1, 1.25]$ of that of Distributed AdamW. The upper-bound is calculated as that the communication of Distributed Muon is 4 (fp32 $\mathbf{G}$ reduce-scatter) + 2 (bf16 Muon gather) + 4 (fp32 $\mathbf{P}$ all-gather), while Distributed AdamW is 4 + 4. In practice, as we usually train with multiple DP, the empirical additional cost usually is closer to the lower-bound 1.[5].
- `Latency.` Distributed Muon has larger end-to-end latencies than Distributed AdamW because it introduces additional communication and requires running Newton-Schulz iteration steps. However, this is not a significant issue because (a) only about 5 Newton-Schultz iteration steps are needed for a good result (discussed in Sec 2.2), and (b) the end-to-end latency caused by the optimizer is negligible compared to the model's forward-backward pass time (e.g. usually 1% to 3%). Moreover, several engineering techniques, such as overlapping gather and computation, and overlapping optimizer reduce-scatter with parameter gather, can further reduce latency.

When training large-scale models in our distributed cluster, Distributed Muon has no noticeable latency overhead compared to its AdamW counterparts. We will soon release a pull request that implements Distributed Muon for the open-source Megatron-LM (Shoeybi et al. 2020) project.

[5]If TP is enabled, Distributed Muon needs an extra bf16 TP gather on TP group.

算法1分布式Muon

需求：完整的梯度 G，DP 分区的动量 m，DP 分区的参数 p，动量 μ。 1: // 在 DP 上进行 Reduce-scatter G 以获得正确的梯度 2: g = reduce_scatter(G, dp_group) 3: // 使用本地分区动量 m 将动量应用于 g 4: g′ = update_with_momentum(g,m μ) 5: // DP Gather：将 g′ 跨 DP 集合成一个完整矩阵 G 6: G = gather(g′, dp_group) 7: // 计算 Muon 更新 8: U = Newton-Schulz(G) 9: // 丢弃 U 的其余部分，仅保留本地分区 u，然后应用更新规则 10: p′ = apply_update(p,u) 11: // 将更新的 p′ all-gather 到 P 12: P = all_gather(p′, dp_group) 13: // 返回更新 RMS 以进行日志记录 14: return $\sqrt{\text{u}^2\text{.mean}()}$

我们提出了一种基于ZeRO-1的分布式Muon解决方案，称为分布式Muon。分布式Muon遵循ZeRO-1，在DP上划分优化器状态，并与普通的Zero-1 AdamW优化器相比引入了两个额外的操作：

1. DP Gather。对于本地DP划分的主权重（$1/DP$模型权重的尺寸），此操作是将相应的划分梯度收集到一个完整的梯度矩阵中。
2. 计算完整更新。在上述收集之后，在完整梯度矩阵上执行牛顿-舒尔茨迭代步骤，如第2.1节所述。请注意，我们将丢弃部分完整更新矩阵，因为我们只需要与本地参数对应的划分部分来执行更新。

分布式Muon的实现描述在算法1中。由分布式Muon引入的附加操作以蓝色显示。

分析我们在几个方面比较了分布式Muon与经典的基于ZeRO-1的分布式AdamW（为简洁起见，简称分布式AdamW）：

- 内存使用。Muon仅使用一个动量缓冲区，而AdamW使用两个动量缓冲区。因此，Muon优化器使用的额外内存是分布式AdamW的一半。
- 通信开销。对于每个设备，额外的DP收集仅由本地DP分区参数$\mathbf{p}$需要。因此，通信成本低于$\mathbf{G}$ 的reduce-scatter或$\mathbf{P}$的全gather。此外，Muon仅需bf16中的Newton-Schulz迭代步骤，从而将通信开销进一步降低至fp32的50%。总体而言，分布式Muon的通信工作量为分布式AdamW的$(1, 1.25]$ 倍。上界计算为分布式Muon的通信为4 (fp32$\mathbf{G}$ reduce-scatter) + 2 (bf16 Muon gather) + 4 (fp32 $\mathbf{P}$ 全gather)，而分布式AdamW为 4 + 4。在实践中，由于我们通常使用多个DP进行训练，经验上的额外成本通常更接近下界1.[5]。
- 延迟。分布式Muon的端到端延迟比分布式AdamW更大，因为它引入了额外的通信，并且需要运行Newton-Schulz迭代步骤。然而，这不是一个重大问题，因为 (a) 仅需大约5个Newton-Schulz迭代步骤即可获得良好结果（如第2.2节所述），以及 (b) 优化器引起的端到端延迟与模型的正向反向传递时间相比微不足道（例如，通常为1%到3%）。此外，重叠收集和计算以及重叠优化器reduce-scatter与参数收集等工程技术可以进一步降低延迟。

在我们分布式集群中训练大规模模型时，分布式Muon与它的AdamW版本相比，没有明显的延迟开销。我们将很快发布一个拉取请求，该请求为开源Megatron-LM（Shoeybi等人，2020）项目实现分布式Muon。

[5]如果启用了TP，Di stributed Muon需要在TP组上执行额外的bf16 TP收集 组。

Table 1: Controlling Muon's Update RMS Across Different Model Params

Methods	Training loss	Validation loss	query weight RMS	MLP weight RMS
Baseline	2.734	2.812	3.586e-2	2.52e-2
Update Norm	**2.72**	**2.789**	4.918e-2	5.01e-2
Adjusted LR	2.721	**2.789**	3.496e-2	4.89e-2

3 Experiments

3.1 Consistent Update RMS

As discussed in Sec 2.2, we aim to match the update RMS across all matrix parameters and also match it with that of AdamW. We experimented with two methods to control the Muon update RMS among parameters and compared them to a baseline that only maintains a consistent RMS with AdamW:

1. `Baseline`. We multiplied the update matrix by $0.2 \cdot \sqrt{H}$ (H is the model hidden size) to maintain a consistent update RMS with AdamW. Note that $\max(A, B)$ equals to H for most matrices.
$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t \cdot \sqrt{H} + \lambda \mathbf{W}_{t-1}) \tag{5}$$
2. `Update Norm`. We can directly normalize the updates calculated via Newton-Schulz iterations so its RMS strictly becomes 0.2;
$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t / \operatorname{RMS}(\mathbf{O}_t) + \lambda \mathbf{W}_{t-1}) \tag{6}$$
3. `Adjusted LR`. For each update matrix, we can scale its learning rate by a factor of $0.2 \cdot \sqrt{\max(A, B)}$ based on its shape.
$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t \cdot \sqrt{\max(A, B)} + \lambda \mathbf{W}_{t-1}) \tag{7}$$

Analysis We designed experiments to illustrate the impact of Muon update RMS at an early training stage, because we observed that unexpected behaviors happened very quickly when training models at larger scale. We experimented with small scale 800M models as described in 3.2. The problem of inconsistent update RMS is more pronounced when the disparity between matrix dimensions increases. To highlight the problem for further study, we slightly modify the model architecture by replacing the Swiglu MLP with a standard 2-layer MLP, changing the shape of its matrix parameters from $[H, 2.6H]$ to $[H, 4H]$. We evaluated the model's loss and monitored a few of its parameters' RMS, specifically, attention query (shape $[H, H]$) and MLP (shape $[H, 4H]$). We evaluated the model after training for 4B tokens out of a 20B-token schedule. From Table 1, we observed several interesting findings:

1. Both `Update Norm` and `Adjusted LR` achieved better performances than `Baseline`;
2. For the MLP weight matrix of shape $[H, 4H]$, both `Update Norm` and `Adjusted LR` obtain a weight RMS that is roughly doubled comparing to `Baseline`. This is reasonable as $\sqrt{\max(H, 4H)}/\sqrt{H} = 2$, so the update RMS of `Update Norm` and `Adjusted LR` is roughly two times of `Baseline`;
3. For the attention query weight matrix of shape $[H, H]$, `Update Norm` still norms the update, while `Adjusted LR` does not because $\sqrt{\max(H, H)}/\sqrt{H} = 1$. As a result, `Adjusted LR` results in a similar weight RMS as `Baseline`, but `Update Norm` has a larger weight rms similar to its MLP.

Based on these findings, we choose the `Adjusted LR` method for future experiments because it has lower cost.

3.2 Scaling Law of Muon

For a fair comparison with AdamW, we performed scaling law experiments on a series of dense models in Llama (Grattafiori et al. 2024) architecture. Building a strong baseline is of crucial importance in optimizer research. Hence, we perform a grid search for hyper-parameters of AdamW, following the compute-optimal training setup (Kaplan et al. 2020) (the grid search experiments can be found in Appendix B). Details of the model architecture and hyper-parameters can be found in Table 2. For Muon, as discussed in Sec 2.2, since we matched Muon's update RMS to AdamW, we directly reused the hyper-parameters that are optimal for the AdamW baseline.

The fitted scaling law curve can be found in figure 3, and the fitted equations are detailed in table 3. As shown in Figure 1a, Muon only requires about 52% training FLOPs to match the performance of AdamW under compute-optimal setting.

表1：控制Muon的更新RMS在不同模型参数下的表现

方法	训练损失	验证损失	查询权重RMS	MLP权重RMS
基线	2.734	2.812	3.586e-2	2.52e-2
更新范数	**2.72**	**2.789**	4.918e-2	5.01e-2
调整学习率	2.721	**2.789**	3.496e-2	4.89e-2

3 个实验

3.1 一致性更新均方根

如第2.2节所述，我们的目标是使所有矩阵参数的更新均方根与AdamW相匹配，同时也要匹配它的均方根。我们实验了两种控制参数间Muon更新均方根的方法，并将它们与仅维护与AdamW一致均方根的基线进行了比较：

1. 基线。我们将更新矩阵乘以 $0.2 \cdot \sqrt{H}$ （H 是模型隐藏层大小）以与 AdamW 维持一致的更新均方根。请注意，对于大多数矩阵，$\max(A, B)$ 等于 H。
$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t \cdot \sqrt{H} + \lambda \mathbf{W}_{t-1}) \tag{5}$$
2. 更新范数。我们可以直接对通过牛顿-舒尔茨迭代计算的更新进行归一化，使其均方根严格变为 0.2；
$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t / \operatorname{RMS}(\mathbf{O}_t) + \lambda \mathbf{W}_{t-1}) \tag{6}$$
3. 调整学习率。对于每个更新矩阵，我们可以根据其形状将其学习率按 $0.2 \cdot \sqrt{\max(A, B)}$ 的因子进行缩放。
$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t \cdot \sqrt{\max(A, B)} + \lambda \mathbf{W}_{t-1}) \tag{7}$$

分析 我们设计了实验来说明 Muon 更新均方根在早期训练阶段的影响，因为我们观察到在更大规模下训练模型时，意外行为会非常快地发生。我们像 3.2 中描述的那样，对 800M 的小规模模型进行了实验。当矩阵维度之间的差异增加时，不一致更新均方根的问题会更加明显。为了进一步研究该问题，我们通过用标准两层 MLP 替换 SwiGLU MLP，略微修改了模型架构，将其矩阵参数的形状从 $[H, 2.6H]$ 改为 $[H, 4H]$。我们在完成 20B-token 计划中的 4B-token 训练后评估了模型的损失，并监控了其几个参数的均方根，特别是注意力查询（形状 $[H, H]$）和 MLP（形状 $[H, 4H]$）。从表1中，我们观察到几个有趣的发现：

1. 更新范数和调整学习率均优于基线；2. 对于形状为 $[H, 4H]$ 的 MLP 权重矩阵，更新范数和调整学习率获得的权重均方根大致是基线的两倍。这是合理的，因为 $\sqrt{\max(H, 4H)}/\sqrt{H} = 2$，所以更新范数和调整学习率的更新均方根大致是基线的两倍；3. 对于形状为 $[H, H]$ 的注意力查询权重矩阵，更新范数仍然对更新进行范数化，而调整学习率不这样做，因为 $\sqrt{\max(H, H)}/\sqrt{H} = 1$。结果，调整学习率的权重均方根与基线相似，而更新范数的权重均方根更大，类似于其 MLP。

基于这些发现，我们选择调整学习率方法进行未来的实验，因为它成本更低。

3.2 Muon的缩放定律

为了与 AdamW 进行公平比较，我们在 Llama (Grattafiori等人 2024) 架构的一系列密集模型上进行了缩放定律实验。在优化器研究中，建立强大的基线至关重要。因此，我们遵循计算最优训练设置（Kaplan等人 2020）对 AdamW 的超参数进行网格搜索（网格搜索实验可以在附录 B 中找到）。模型架构和超参数的详细信息可以在表 2 中找到。对于 Muon，如第 2.2 节所述，由于我们将 Muon 的更新均方根与 AdamW 匹配，我们直接重用了对 AdamW 基线最优的超参数。

拟合的缩放律曲线可以在图3中找到，拟合的方程式在表3中详细说明。如图1a所示，Muon仅需约52%的训练FLOPs即可在计算最优设置下匹配AdamW的性能。

Table 2: Scaling Law Models and Hyper-Parameters

# Params. w/o Embedding	Head	Layer	Hidden	Tokens	LR	Batch Size*
399M	12	12	1536	8.92B	9.503e-4	96
545M	14	14	1792	14.04B	9.143e-4	128
822M	16	16	2048	20.76B	8.825e-4	160
1.1B	18	18	2304	28.54B	8.561e-4	192
1.5B	20	20	2560	38.91B	8.305e-4	256

*In terms of number of examples in 8K context length.

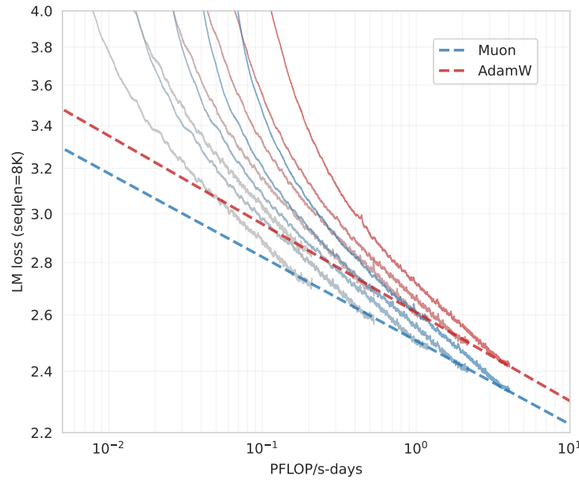


Figure 3: Fitted scaling law curves for Muon and AdamW optimizers.

3.3 Pretraining with Muon

Model Architecture To evaluate Muon against contemporary model architectures, we pretrained from scratch using the deepseek-v3-small architecture (DeepSeek-AI et al. 2024) as it demonstrates strong performance and the original results serve as a reference for comparison. Our pretrained model has 2.24B activated and 15.29B total parameters (3B activated and 16B total when including embedding). Minor modifications to the architecture are detailed in Appendix C.

Pretraining Data Our pretraining data details can be found in K. Team 2025. The maximum context length during pretraining is 8K.

Pretraining The model is trained in several stages. We use a 1e-3 auxfree bias update rate in stage 1 and 2, and 0.0 auxfree bias update rate in stage 3. The weight decay is set to 0.1 for all stages. More details and discussions of model training can be found in the Appendix D.

1. `0 to 33B tokens`: In this stage, the learning rate linearly increases to 4.2e-4 in 2k steps. The batch size is kept at 2048 examples;

Table 3: Fitted parameters of the scaling law curves

	Muon	AdamW
LM loss (seqlen=8K)	$2.506 \times C^{-0.052}$	$2.608 \times C^{-0.054}$

表 2：缩放律模型和超参数

# 无嵌入的参数	Head	层	隐藏	令牌	LR	批处理大小*
399M	12	12	1536	8.92B	9.503e-4	96
545M	14	14	1792	14.04B	9.143e-4	128
822M	16	16	2048	20.76B	8.825e-4	160
1.1B	18	18	2304	28.54B	8.561e-4	192
1.5B	20	20	2560	38.91B	8.305e-4	256

在8K上下文长度的情况下，示例数量方面。

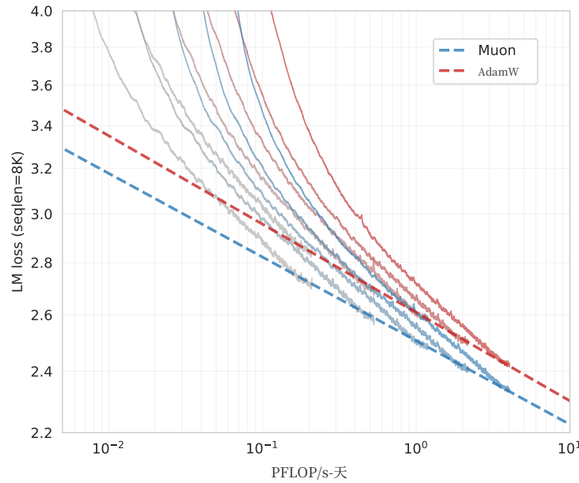


图3：Muon和AdamW优化器的拟合缩放律曲线。

3.3 使用Muon进行预训练

模型架构为了评估Muon与当代模型架构的对比，我们使用deepseek-v3-small架构（DeepSeek-AI等，2024年）从头开始预训练，因为它表现出色，原始结果作为比较的参考。我们的预训练模型有2.24B个激活参数和15.29B个总参数（包含嵌入时为3B个激活参数和16B个总参数）。对架构的微小修改在附录C中详细说明。

预训练数据我们的预训练数据细节可以在 K. Team 2025 中找到。预训练时的最大上下文长度为 8K。

预训练模型在多个阶段进行训练。我们在第一阶段和第二阶段使用 1e-3 的 auxfree 偏差更新率，在第三阶段使用 0.0 的 auxfree 偏差更新率。权重衰减对所有阶段均设置为 0.1。更多关于模型训练的细节和讨论可以在附录 D 中找到。

1. 0到33B个token：在这个阶段，学习率在2k步内线性增加到4.2e-4。批大小保持在2048个示例；

表 3：缩放律曲线的拟合参数

	Muon	AdamW
LM 损失（seqlen=8K）	$2.506 \times C^{-0.052}$	$2.608 \times C^{-0.054}$

2. `33B to 5.2T tokens`: In this stage, the learning rate decays from 4.2e-4 to 4.2e-5 in a cosine style. We keep the batch size at 2048 until 200B tokens, and then doubled to 4096 for the remaining;
3. `5.2T to 5.7T tokens`: In this stage (also referred as the cooldown stage), the learning rate increases to 1e-4 in in 100 steps, and then linearly decays to 0 in 500B tokens, and we keep a constant 4096 batch size. In this stage, we use the highest quality data, focusing on math, code, and reasoning.

Evaluation Benchmarks Our evaluation encompasses four primary categories of benchmarks, each designed to assess distinct capabilities of the model:

- **English Language Understanding and Reasoning**: MMLU(5-shot)(Hendrycks, Burns, Basart, et al. 2021), MMLU-pro(5-shot) (Wang et al. 2024), BBH(3-shot) (Suzgun et al. 2022), TriviaQA(5-shot) (Joshi et al. 2017)
- **Code Generation**: HumanEval(pass@1) (M. Chen et al. 2021), MBPP(pass@1)(Austin et al. 2021)
- **Mathematical Reasoning**: GSM8K(4-shot) (Cobbe et al. 2021) MATH (Hendrycks, Burns, Kadavath, et al. 2021), CMATH (Wei et al. 2023)
- **Chinese Language Understanding and Reasoning**: C-Eval(5-shot) (Y. Huang et al. 2023), CMMLU(5-shot)(H. Li et al. 2024)

Performance We named our model trained with Muon "Moonlight". We compared Moonlight with different public models on a similar scale. We first evaluated Moonlight at 1.2T tokens and compared it with the following models that have the same architecture and trained with comparable number of tokens:

- `Deepseek-v3-Small` (DeepSeek-AI et al. 2024) is a 2.4B/16B-parameter MoE model trained with 1.33T tokens;
- `Moonlight-A` follows the same training settings as Moonlight, except that it uses the AdamW optimizer.

For Moonlight and Moonlight-A, we used the intermediate 1.2T token checkpoint of the total 5.7T pretraining, where the learning rate is not decayed to minimal and the model has not gone through the cooldown stage yet.

Table 4: Comparison of different models at around 1.2T tokens.

	Benchmark (Metric)	DSV3-Small	Moonlight-A@1.2T	Moonlight@1.2T
	Activated Params†	2.24B	2.24B	2.24B
	Total Params†	15.29B	15.29B	15.29B
	Training Tokens	1.33T	1.2T	1.2T
	Optimizer	AdamW	AdamW	Muon
English	MMLU	53.3	60.2	**60.4**
	MMLU-pro	-	26.8	**28.1**
	BBH	41.4	**45.3**	43.2
	TriviaQA	-	57.4	**58.1**
Code	HumanEval	26.8	29.3	**37.2**
	MBPP	36.8	49.2	**52.9**
Math	GSM8K	31.4	43.8	**45.0**
	MATH	10.7	16.1	**19.8**
	CMath	-	57.8	**60.2**
Chinese	C-Eval	-	57.2	**59.9**
	CMMLU	-	58.2	**58.8**

† The reported parameter counts exclude the embedding parameters.

As shown in Table 4, Moonlight-A, our AdamW-trained baseline model, demonstrates strong performance compared to similar public models. Moonlight performs significantly better than Moonlight-A, proving the scaling effectiveness of Muon. We observed that Muon especially excels on Math and Code related tasks, and we encourage the research community to further investigate this phenomena. After Moonlight is fully trained to 5.7T tokens, we compared it with public models at similar scale and showed the results in Table 5:

- `LLAMA3-3B` from Grattafiori et al. 2024 is a 3B-parameter dense model trained with 9T tokens.
- `Qwen2.5-3B` from Yang et al. 2024 is a 3B-parameter dense model trained with 18T tokens.

2. 33B到5.2T个token：在这个阶段，学习率以余弦方式从4.2e-4衰减到4.2e-5。我们保持批大小为2048，直到200B个token，然后增加到4096以完成剩余部分；3. 5.2T到5.7T个token：在这个阶段（也称为冷却阶段），学习率在100步内增加到1e-4，然后线性衰减到0，在500B个token内，我们保持4096的恒定批大小。在这个阶段，我们使用最高质量的数据，专注于数学、代码和推理。

评估基准 我们的评估包含四个主要类别的基准，每个基准旨在评估模型的独特能力：

- **英语语言理解和推理**：MMLU(5-shot)(Hendrycks, Burns, Basart, 等 2021)，MMLU-pro(5-shot) (Wang 等 2024)，BBH(3-shot) (Suzgun 等 2022)，TriviaQA(5-shot) (Joshi 等 2017)
- **代码生成**：HumanEval(pass@1) (M. Chen 等 2021)，MBPP(pass@1)(Austin 等 2021)
- **数学推理**：GSM8K(4-shot) (Cobbe 等 2021) MATH (Hendrycks, Burns, Kadavath, 等 2021)，CMATH (Wei 等 2023)
- **中文语言理解与推理**: C-Eval(5-shot) (Y. Huang 等人 2023), CMMLU(5-shot)(H. Li 等人 2024)

性能我们使用Muon训练的模型命名为"Moonlight"。我们在相似规模上比较了Moonlight与不同的公开模型。我们首先在1.2T token上评估Moonlight，并将其与以下具有相同架构且使用可比token数量的模型进行比较：

- Deepseek-v3-Small (DeepSeek-AI 等人 2024) 是一个2.4B/16B参数的MoE模型，使用1.33T token进行训练；
- Moonlight-A遵循Moonlight相同的训练设置，只是它使用了AdamW优化器。

对于Moonlight和Moonlight-A，我们使用了总共5.7T预训练中的中间1.2T token检查点，其中学习率尚未衰减到最小，模型也尚未经过冷却阶段。

表4：不同模型在约1.2T token下的比较。

	基准（指标）	DSV3-Small	Moonlight-A@1.2T	Moonlight@1.2T
	激活参数†	2.24B	2.24B	2.24B
	总参数†	15.29B	15.29B	15.29B
	训练令牌	1.33T	1.2T	1.2T
	优化器	AdamW	AdamW	Muon
英语	MMLU	53.3	60.2	**60.4**
	MMLU-pro	-	26.8	**28.1**
	BBH	41.4	**45.3**	43.2
	TriviaQA	-	57.4	**58.1**
Code	HumanEval	26.8	29.3	**37.2**
	MBPP	36.8	49.2	**52.9**
Math	GSM8K	31.4	43.8	**45.0**
	MATH	10.7	16.1	**19.8**
	CMath	-	57.8	**60.2**
中文	C-Eval	-	57.2	**59.9**
	CMMLU	-	58.2	**58.8**

† 报告的参数数量不包括嵌入参数。

如表4所示，Moonlight-A，我们用AdamW训练的基线模型，与类似的公开模型相比表现出色。Moonlight的表现明显优于Moonlight-A，证明了Muon的扩展有效性。我们观察到Muon在数学和代码相关任务上尤其出色，并鼓励研究社区进一步研究这一现象。在Moonlight完全训练到5.7T个token后，我们将其与规模相似的公开模型进行了比较，结果如表5所示：

- Grattafiori等人2024年的LLAMA3-3B是一个3B参数密集模型，用9T个token训练。
- Yang等人2024年的Qwen2.5-3B是一款3B参数密集模型，使用18T个token进行训练。

Table 5: Comparison of different models on various benchmarks.

	Benchmark (Metric)	Llama3.2-3B	Qwen2.5-3B	DSV2-Lite	Moonlight
	Activated Param†	2.81B	2.77B	2.24B	2.24B
	Total Params†	2.81B	2.77B	15.29B	15.29B
	Training Tokens	9T	18T	5.7T	5.7T
	Optimizer	AdamW	Unknown	AdamW	Muon
English	MMLU	54.7	65.6	58.3	**70.0**
	MMLU-pro	25.0	34.6	25.5	**42.4**
	BBH	46.8	56.3	44.1	**65.2**
	TriviaQA‡	59.6	51.1	65.1	**66.3**
Code	HumanEval	28.0	42.1	29.9	**48.1**
	MBPP	48.7	57.1	43.2	**63.8**
Math	GSM8K	34.0	**79.1**	41.1	77.4
	MATH	8.5	42.6	17.1	**45.3**
	CMath	-	80.0	58.4	**81.1**
Chinese	C-Eval	-	75.0	60.3	**77.2**
	CMMLU	-	75.0	64.3	**78.2**

† The reported parameter counts exclude the embedding parameters.‡ We tested all listed models with the full set of TriviaQA.

- `Deepseek-v2-Lite` from DeepSeek-AI 2024 is a 2.4B/16B-parameter MOE model trained with 5.7T tokens.

As shown in Table 5, Moonlight outperforms models with similar architectures trained with an equivalent number of tokens. Even when compared to dense models trained on substantially larger datasets, Moonlight maintains competitive performance. Detailed comparisons can be found in Appendix E. The performance of Moonlight is further compared with other well-known language models on MMLU and GSM8k, as illustrated in Figure 1b and Appendix E Figure 8.[6]. Notably, Moonlight lies on the Pareto frontier of model performance versus training budget, outperforming many other models across various sizes.

3.4 Dynamics of Singular Spectrum

In order to validate the intuition that Muon can optimize the weight matrices in more diverse directions, we conducted a spectral analysis of the weight matrices trained with Muon and AdamW. For a weight matrix with singular values $\sigma = (\sigma_1, \sigma_2, \cdots, \sigma_n)$, we calculate the SVD entropy (Alter et al. 2000; Roy et al. 2007) of this matrix as follows:

$$H(\sigma) = -\frac{1}{\log n}\sum_{i=1}^{n}\frac{\sigma_i^2}{\sum_{j=1}^{n}\sigma_j^2}\log\frac{\sigma_i^2}{\sum_{j=1}^{n}\sigma_j^2}$$

As shown in Figure 4, we visualized the average SVD entropy of the weight matrices across different training checkpoints during pretraining with 1.2T tokens. We can see that across all training checkpoints and all groups of weight matrices, the SVD entropy of Muon is higher than that of AdamW, which verifies the intuition that Muon can provide a more diverse spectrum of updates for the weight matrices. This discrepancy is more significant in the router weights for expert selection, which indicates that mixture-of-expert models can benefit more from Muon.

Moreover, we visualized the singular value distributions of each weight matrix at the checkpoint trained with 1.2T tokens as demonstrated in Appendix F. We find that, for over 90% of the weight matrices, the SVD entropy when optimized by Muon is higher than that of AdamW, providing strong empirical evidence for Muon's superior capability in exploring diverse optimization directions.

3.5 Supervised Finetuning (SFT) with Muon

In this section, we present ablation studies on the Muon optimizer within the standard SFT stage of LLM training. Our findings demonstrate that the benefits introduced by Muon persist during the SFT stage. Specifically, a model that is both Muon-pretrained and Muon-finetuned outperforms others in the ablation studies. However, we also observe that when the SFT optimizer differs from the pretraining optimizer, SFT with Muon does not show a significant advantage over AdamW. This suggests that there is still considerable room for further exploration, which we leave for future work.

[6]Performance metrics and computational requirements (FLOPs) for baseline models are sourced from (OLMo et al. 2024)

表 5：不同模型在各种基准上的比较。

	基准（指标）	Llama3.2-3B	Qwen2.5-3B	DSV2-Lite	Moonlight
	激活参数†	2.81B	2.77B	2.24B	2.24B
	总参数†	2.81B	2.77B	15.29B	15.29B
	训练令牌	9T	18T	5.7T	5.7T
	优化器	AdamW	未知	AdamW	Muon
英语	MMLU	54.7	65.6	58.3	**70.0**
	MMLU-pro	25.0	34.6	25.5	**42.4**
	BBH	46.8	56.3	44.1	**65.2**
	TriviaQA‡	59.6	51.1	65.1	**66.3**
Code	HumanEval	28.0	42.1	29.9	**48.1**
	MBPP	48.7	57.1	43.2	**63.8**
Math	GSM8K	34.0	**79.1**	41.1	77.4
	MATH	8.5	42.6	17.1	**45.3**
	CMath	-	80.0	58.4	**81.1**
中文	C-Eval	-	75.0	60.3	**77.2**
	CMMLU	-	75.0	64.3	**78.2**

† 据报道的参数 计数单位不包括嵌入参数.‡ 我们使用完整集测试了所有列出的模型。

A.

- 来自DeepSeek-AI的2024年Deepseek-V2-Lite是一个2.4B/16B参数的MOE模型，使用5.7T个token进行训练。

如表5所示，Moonlight在架构相似的模型中表现更优，且使用相同数量的token进行训练。即使与在规模显著更大的数据集上训练的密集模型相比，Moonlight仍保持有竞争力的性能。详细对比可参考附录E。Moonlight的性能还与其他知名语言模型在MMLU和GSM8k上的表现进行了对比，如图1b和附录E图8所示。[6]值得注意的是，Moonlight位于模型性能与训练预算的帕累托前沿，在各种规模上均优于许多其他模型。

3.4 奇异谱的动力学

为了验证 Muon 可以在更多样化的方向上优化权重矩阵的直觉，我们对使用 Muon 和 AdamW 训练的权重矩阵进行了谱分析。对于一个具有奇异值 $\sigma = (\sigma_1, \sigma_2, \cdots, \sigma_n)$ 的权重矩阵，我们按照如下方式计算该矩阵的 SVD 熵（Alter 等人，2000；Roy 等人，2007）：

$$H(\sigma) = -\frac{1}{\log n}\sum_{i=1}^{n}\frac{\sigma_i^2}{\sum_{j=1}^{n}\sigma_j^2}\log\frac{\sigma_i^2}{\sum_{j=1}^{n}\sigma_j^2}$$

如图4所示，我们可视化了在用1.2T tokens进行预训练时，不同训练检查点下权重矩阵的平均SVD熵。我们可以看到，在所有训练检查点和所有权重矩阵组中，Muon的SVD熵都高于AdamW，这验证了Muon可以为权重矩阵提供更多样化的更新谱的直觉。这种差异在用于专家选择的路由权重中更为显著，这表明混合专家模型能从Muon中获益更多。

此外，我们可视化了使用1.2T token训练的检查点中每个权重矩阵的奇异值分布，如附录F所示。我们发现，对于超过90%的权重矩阵，当使用Muon进行优化时，其SVD熵高于AdamW，为Muon在探索多样化优化方向方面具有卓越能力提供了强有力的经验证据。

3.5 基于Muon的监督微调 (SFT)

在本节中，我们针对大语言模型训练的标准SFT阶段中的Muon优化器进行了消融研究。我们的研究发现，Muon带来的优势在SFT阶段仍然持续。具体来说，一个同时经过Muon预训练和Muon微调的模型在消融研究中表现优于其他模型。然而，我们也观察到，当SFT优化器与预训练优化器不同时，使用Muon的SFT并不比AdamW表现出显著优势。这表明仍有相当大的空间可供进一步探索，我们将其留作未来工作。

[6]性能指标和计算需求（FLOPs）对于基线模型是 sourced from (OLMo et al. 2024)

{v1}Performance metrics and computational requirements (FLOPs) for baseline models are

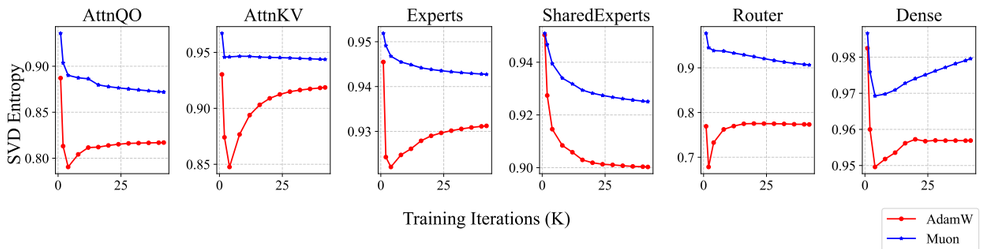


Figure 4: SVD entropy of weight matrices across different training iterations. We categorize the weight matrices into 6 different groups: 1) AttnQO denotes the weight matrices related to the query and output projection in the attention layer; 2) AttnKV denotes the weight matrices related to the key and value projection in the attention layer; 3) Experts denotes the weight matrices in expert models; 4) SharedExperts denotes the weight matrices in shared expert models; 5) Router denotes the weight matrices in the router; 6) Dense denotes the weight matrices in the first dense layer. The SVD entropy is calculated as the macro-average of the weight matrices in each group across all layers. For weights in expert models, we only calculate 3 out of 64 experts in different layers for efficiency.

3.5.1 Ablation Studies on the Interchangeability of Pretrain and SFT Optimizers

To further investigate Muon's potential, we finetuned Moonlight@1.2T and Moonlight-A@1.2T using both the Muon and AdamW optimizers. These models were finetuned for two epochs on the open-source tulu-3-sft-mixture dataset (Lambert et al. 2024), which contains 4k sequence length data. The learning rate followed a linear decay schedule, starting at 5×10^{-5} and gradually reducing to 0. The results, shown in Table 6, highlight the superior performance of Moonlight@1.2T compared to Moonlight-A@1.2T.

Table 6: Examining the impact of optimizer interchangeability between pretraining and SFT phases.

Benchmark (Metric)	**# Shots**	**Moonlight-1.2T**			
Pretraining Optimizer	-	Muon	AdamW	Muon	AdamW
SFT Optimzier	-	Muon	Muon	AdamW	AdamW
MMLU (EM)	0-shot (CoT)	**55.7**	55.3	50.2	52.0
HumanEval (Pass@1)	0-shot	**57.3**	53.7	52.4	53.1
MBPP (Pass@1)	0-shot	**55.6**	55.5	55.2	55.2
GSM8K (EM)	5-shot	**68.0**	62.1	64.9	64.6

3.5.2 SFT with Muon on public pretrained models

We further applied Muon to the supervised fine-tuning (SFT) of a public pretrained model, specifically the Qwen2.5-7B base model (Yang et al. 2024), using the open-source tulu-3-sft-mixture dataset (Lambert et al. 2024). The dataset was packed with an 8k sequence length, and we employed a cosine decay learning rate schedule, starting at 2×10^{-5} and gradually decreasing to 2×10^{-6}. The results are presented in Table 7. For comparison, we show that the Muon-finetuned model achieves performance on par with the Adam-finetuned model. These results indicate that for optimal performance, it is more effective to apply Muon during the pretraining phase rather than during supervised fine-tuning.

Table 7: Comparison of Adam and Muon optimizers applied to the SFT of the Qwen2.5-7B pretrained model.

Benchmark (Metric)	**# Shots**	**Adam-SFT**	**Muon-SFT**
Pretrained Model	-	Qwen2.5-7B	
MMLU (EM)	0-shot (CoT)	**71.4**	70.8
HumanEval (Pass@1)	0-shot	**79.3**	77.4
MBPP (Pass@1)	0-shot	**71.9**	71.6
GSM8K (EM)	5-shot	**89.8**	85.8

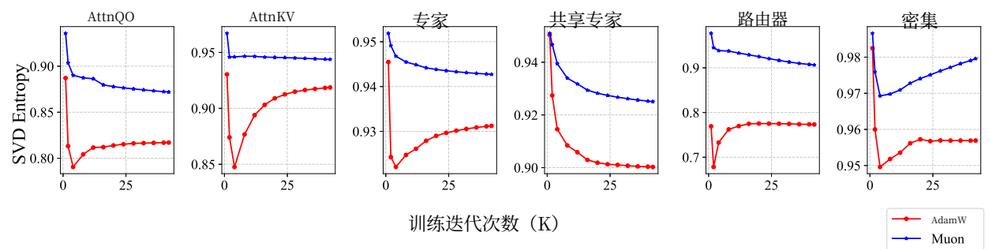


图 4：不同训练迭代次数下权重矩阵的 SVD 摘要。我们将权重矩阵分为 6 组：1) AttnQO 表示注意力层中与查询和输出投影相关的权重矩阵；2) AttnKV 表示注意力层中与键和值投影相关的权重矩阵；3) Experts 表示专家模型中的权重矩阵；4) SharedExperts 表示共享专家模型中的权重矩阵；5) Router 表示路由器中的权重矩阵；6) Dense 表示第一层密集层中的权重矩阵。SVD 摘要计算为所有层中每个组的权重矩阵的宏平均值。对于专家模型中的权重，我们出于效率考虑，仅在每层计算 64 个专家中的 3 个。

3.5.1 预训练和 SFT 优化器可互换性的消融研究

为了进一步探究Muon的潜力，我们使用Muon和AdamW优化器对Moonlight@1.2T和Moonlight-A@1.2T进行了微调。这些模型在开源tulu-3-sft-mixture数据集（Lambert等人，2024）上进行了两个epoch的微调，该数据集包含4k序列长度的数据。学习率遵循线性衰减策略，从 5×10^{-5} 开始逐渐减少至0。结果如表6所示，突显了Moonlight@1.2T相比Moonlight-A@1.2T的优越性能。

Table 6：检查预训练和SFT阶段之间优化器可互换性的影响 s.

基准（指标）	**# 射击次数**	**Moonlight-1.2T**			
预训练优化器	-	Muon	AdamW	Muon	AdamW
SFT优化器	-	Muon	Muon	AdamW	AdamW
MMLU（EM）	0-shot（CoT）	**55.7**	55.3	50.2	52.0
HumanEval（Pass@1）	0-shot	**57.3**	53.7	52.4	53.1
MBPP（Pass@1）	0-shot	**55.6**	55.5	55.2	55.2
GSM8K（EM）	5-shot	**68.0**	62.1	64.9	64.6

3.5.2 使用Muon在公开预训练模型上进行SFT

我们进一步将Muon应用于公共预训练模型的监督微调（SFT），具体是Qwen2.5-7B基础模型（Yang等人2024），并使用开源tulu-3-sft-mixture数据集（Lambert等人2024）。该数据集打包了8k序列长度，我们采用了余弦衰减学习率调度，从 2×10^{-5}开始，逐渐降低至 2×10^{-6}。结果展示在表7中。为了比较，我们展示了Muon微调模型在性能上与Adam微调模型相当。这些结果表明，为了获得最佳性能，在预训练阶段应用Muon比在监督微调阶段应用更有效。

表7：Adam和Muon优化器应用于Qwen2.5-7B预训练模型SFT的比较

基准（指标）	**样本数**	**Adam-SFT**	**Muon-SFT**
预训练模型	-	Qwen2.5-7B	
MMLU（EM）	0-shot（CoT）	**71.4**	70.8
HumanEval（Pass@1）	0-shot	**79.3**	77.4
MBPP（Pass@1）	0-shot	**71.9**	71.6
GSM8K（EM）	5-shot	**89.8**	85.8

4 Discussions

There are several possible directions for future research that could further explore and expand upon the current findings.

Incorporating All Parameters into the Muon Framework Currently, the Muon optimizer is utilized in conjunction with the Adam optimizer, where certain parameters remain under the purview of Adam optimization. This hybrid approach, while functional, presents an opportunity for improvement. The integration of the optimization of all parameters exclusively within the Muon framework is a topic of significant research interest.

Extending Muon to Schatten Norms The Muon optimizer can be interpreted as the steepest descent method under the spectral norm. Given the broad applicability and versatility of Schatten norms, extending Muon to encompass the general Schatten norm is a promising direction. This extension may unlock additional optimization capabilities and potentially yield superior results compared to the current spectral norm-based implementation.

Understanding and Solving the Pretraining-Finetuning Mismatch A notable phenomenon observed in practice is the suboptimal performance of models pretrained with AdamW when fine-tuned with Muon, and vice versa. This optimizer mismatch presents a significant barrier to effectively leveraging the extensive repository of AdamW-pretrained checkpoints, thereby necessitating a rigorous theoretical investigation. A precise understanding of the underlying mechanisms is essential for devising robust and effective solutions.

5 Conclusions

In this technical report, we presented a comprehensive study on the scalability of Muon in LLM training. Through systematic analysis and improvements, we successfully applied Muon to a 3B/16B-parameter MoE model trained on 5.7 trillion tokens. Our results demonstrate that Muon can effectively replace AdamW as the standard optimizer for large-scale LLM training, offering significant advantages in both training efficiency and model performance. By open-sourcing our implementation, the Moonlight model, and intermediate training checkpoints, we aim to facilitate further research in scalable optimization techniques and accelerate the development of training methods for LLMs.

4 讨论话题

存在几种可能性 为未来进一步探索和扩展当前发现的未来研究方向 .

将所有参数集成到 Muon 框架中 目前，Muon 优化器与 Adam 优化器结合使用，其中某些参数仍由 Adam 优化器管理。这种混合方法虽然有效，但存在改进的机会。将所有参数的优化完全集成到 Muon 框架中是一个重要的研究方向。

将 Muon 扩展到 Schatten 范数 可解释为 Muon 优化器是在谱范数下的最速下降法。鉴于 Schattennorms 的广泛适用性和多功能性，将 Muon 扩展到包含一般 Schatten 范数是一个有前景的方向。这种扩展可能会解锁额外的优化能力，并且与当前的基于谱范数的实现相比，可能产生更优的结果。

理解和解决预训练-微调不匹配问题在实践中观察到的一个显著现象是，使用AdamW预训练的模型在用Muon微调时表现不佳，反之亦然。这种优化器不匹配为有效利用AdamW预训练的检查点库构成了一道重大障碍，因此需要进行严谨的理论研究。为了制定稳健有效的解决方案，对潜在机制进行精确理解至关重要。

5 结论

在本技术报告中，我们全面研究了 Muon 在大语言模型训练中的可扩展性。通过系统分析和改进，我们成功地将 Muon 应用于一个 3B/16B 参数的 MoE 模型，该模型在 5.7 万亿个 token 上进行了训练。我们的结果表明，Muon 可以有效地替代 AdamW 作为大规模大语言模型训练的标准优化器，在训练效率和模型性能方面都提供了显著的优势。通过开源我们的实现、Moonlight 模型以及中间训练检查点，我们旨在促进可扩展优化技术的研究，并加速大语言模型训练方法的发展。

References

Alter, Orly, Patrick O. Brown, and David Botstein. "Singular value decomposition for genome-wide expression data processing and modeling". In: *Proceedings of the National Academy of Sciences* 97.18 (2000), pp. 10101–10106. DOI: 10.1073/pnas.97.18.10101. eprint: https://www.pnas.org/doi/pdf/10.1073/pnas.97.18.10101. URL: https://www.pnas.org/doi/abs/10.1073/pnas.97.18.10101.

Austin, Jacob et al. *Program Synthesis with Large Language Models*. 2021. arXiv: 2108.07732 [cs.PL]. URL: https://arxiv.org/abs/2108.07732.

Bernstein, Jeremy and Laker Newhouse. *Old Optimizer, New Norm: An Anthology*. 2024. arXiv: 2409.20325 [cs.LG]. URL: https://arxiv.org/abs/2409.20325.

Bi, Xiao et al. "Deepseek llm: Scaling open-source language models with longtermism". In: *arXiv preprint arXiv:2401.02954* (2024).

Cesista, Franz Louis. *Deep Learning Optimizers as Steepest Descent in Normed Spaces*. 2024. URL: http://leloykun.github.io/ponder/steepest-descent-opt/.

Chen, Mark et al. "Evaluating Large Language Models Trained on Code". In: (2021). arXiv: 2107.03374 [cs.LG].

Cobbe, Karl et al. *Training Verifiers to Solve Math Word Problems*. 2021. arXiv: 2110.14168 [cs.LG]. URL: https://arxiv.org/abs/2110.14168.

DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. arXiv: 2405.04434 [cs.CL].

DeepSeek-AI et al. *DeepSeek-V3 Technical Report*. 2024. arXiv: 2412.19437 [cs.CL]. URL: https://arxiv.org/abs/2412.19437.

Franz, Louis Cesista. *The Case for Muon*. Oct. 2024. URL: https://x.com/leloykun/status/1846842887839125941 (visited on 02/18/2025).

Grattafiori, Aaron et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: https://arxiv.org/abs/2407.21783.

Hendrycks, Dan, Collin Burns, Steven Basart, et al. *Measuring Massive Multitask Language Understanding*. 2021. arXiv: 2009.03300 [cs.CY]. URL: https://arxiv.org/abs/2009.03300.

Hendrycks, Dan, Collin Burns, Saurav Kadavath, et al. *Measuring Mathematical Problem Solving With the MATH Dataset*. 2021. arXiv: 2103.03874 [cs.LG]. URL: https://arxiv.org/abs/2103.03874.

Hoffmann, Jordan et al. *Training Compute-Optimal Large Language Models*. 2022. arXiv: 2203.15556 [cs.CL]. URL: https://arxiv.org/abs/2203.15556.

Huang, Yuzhen et al. *C-Eval: A Multi-Level Multi-Discipline Chinese Evaluation Suite for Foundation Models*. 2023. arXiv: 2305.08322 [cs.CL]. URL: https://arxiv.org/abs/2305.08322.

Jordan, Keller et al. *Muon: An optimizer for hidden layers in neural networks*. 2024. URL: https://kellerjordan.github.io/posts/muon/.

Joshi, Mandar et al. *TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension*. 2017. arXiv: 1705.03551 [cs.CL]. URL: https://arxiv.org/abs/1705.03551.

Kaplan, Jared et al. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 [cs.LG]. URL: https://arxiv.org/abs/2001.08361.

Kingma, Diederik P. and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2015. URL: http://arxiv.org/abs/1412.6980.

Lambert, Nathan et al. "Tülu 3: Pushing Frontiers in Open Language Model Post-Training". In: (2024).

Li, Haonan et al. *CMMLU: Measuring massive multitask language understanding in Chinese*. 2024. arXiv: 2306.09212 [cs.CL]. URL: https://arxiv.org/abs/2306.09212.

Li, Xi-Lin. "Preconditioned Stochastic Gradient Descent". In: *IEEE Transactions on Neural Networks and Learning Systems* 29.5 (May 2018), pp. 1454–1466. ISSN: 2162-2388. DOI: 10.1109/tnnls.2017.2672978. URL: http://dx.doi.org/10.1109/TNNLS.2017.2672978.

– *Preconditioner on Matrix Lie Group for SGD*. 2018. arXiv: 1809.10232 [stat.ML]. URL: https://arxiv.org/abs/1809.10232.

– *Stochastic Hessian Fittings with Lie Groups*. 2024. arXiv: 2402.11858 [stat.ML]. URL: https://arxiv.org/abs/2402.11858.

Li, Xilin. *Black Box Lie Group Preconditioners for SGD*. 2022. arXiv: 2211.04422 [stat.ML]. URL: https://arxiv.org/abs/2211.04422.

Liu, Hong et al. "Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training". In: *The Twelfth International Conference on Learning Representations*. 2024. URL: https://openreview.net/forum?id=3xHDeA8Noi.

Loshchilov, Ilya and Frank Hutter. "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=Bkg6RiCqY7.

参考文献

Alter、Orly、Patrick O. Brown 和 David Botstein。"奇异值分解用于全基因组表达数据处理和建模"。发表于《美国国家科学院院报》97.18（2000年），第 10101–10106 页。DOI：10.1073/pnas.97.18.10101。电子版：https://www.pnas.org/doi/pdf/10.1073/pnas.97.18.10101。网址：https://www.pnas.org/doi/abs/10.1073/pnas.97.18.10101。Austin、Jacob 等人。使用大语言模型的程序合成。2021年。arXiv：2108.07732 [cs.PL]。网址：https://arxiv.org/abs/2108.07732。Bernstein、Jeremy 和 Laker Newhouse。《旧优化器，新范数：文集》。2024年。arXiv：2409.20325 [cs.LG]。网址：https://arxiv.org/abs/2409.20325。Bi、Xiao 等人。"Deepseek llm：使用长期主义扩展开源语言模型"。发表于 arXiv 预印本 arXiv:2401.02954（2024年）。Cesista、Franz Louis。《作为范数空间中最速下降的深度学习优化器》。2024年。网址：http://leloykun.github.io/ponder/steepest-descent-opt/。Chen、Mark 等人。"评估在代码上训练的大型语言模型"。发表于（2021年）。arXiv：2107.03374 [cs.LG]。Cobbe、Karl 等人。训练验证器解决数学文字问题。2021年。arXiv：2110.14168 [cs.LG]。网址：https://arxiv.org/abs/2110.14168。DeepSeek-AI。《DeepSeek-V2：一个强大、经济高效且高效的专家混合语言模型》。2024年。arXiv：2405.04434 [cs.CL]。DeepSeek-AI 等人。《DeepSeek-V3 技术报告》。2024年。arXiv：2412.19437 [cs.CL]。网址：https://arxiv.org/abs/2412.19437。Franz、Louis Cesista。《支持 Muon 的案例》。2024年10月。网址：https://x.com/leloykun/status/1846842887839125941（访问于 2025 年 02 月 18 日）。Grattafiori、Aaron 等人。《Llama 3 模型群》。2024年。arXiv：2407.21783 [cs.AI]。网址：https://arxiv.org/abs/2407.21783。Hendrycks、Dan、Collin Burns、Steven Basart 等人。测量大规模多任务语言理解。2021年。arXiv：2009.03300 [cs.CY]。网址：https://arxiv.org/abs/2009.03300。Hendrycks、Dan、Collin Burns、Saurav Kadavath 等人。使用 MATH 数据集测量数学问题解决能力。2021年。arXiv：2103.03874 [cs.LG]。网址：https://arxiv.org/abs/2103.03874。Hoffmann、Jordan 等人。训练计算最优的大语言模型。2022年。arXiv：2203.15556 [cs.CL]。网址：https://arxiv.org/abs/2203.15556。Huang、Yuzhen 等人。《C-Eval：为基础模型的多级多学科中文评估套件》。2023年。arXiv：2305.08322 [cs.CL]。网址：https://arxiv.org/abs/2305.08322。Jordan、Keller 等人。《Muon：神经网络隐藏层的优化器》。2024年。网址：https://kellerjordan.github.io/posts/muon/。Joshi、Mandar 等人。《TriviaQA：一个用于阅读理解的远程监督大规模挑战数据集》。2017年。arXiv：1705.03551 [cs.CL]。网址：https://arxiv.org/abs/1705.03551。Kaplan、Jared 等人。《神经语言模型的规模定律》。2020年。arXiv：2001.08361 [cs.LG]。网址：https://arxiv.org/abs/2001.08361。Kingma、Diederik P. 和 Jimmy Ba。"Adam：一种随机优化的方法"。发表于第 3 届国际学习表示会议，ICLR 2015，美国加利福尼亚州圣地亚哥，2015年5月7-9日，会议论文集。由 Yoshua Bengio 和 Yann LeCun 编辑。2015年。网址：http://arxiv.org/abs/1412.6980。Lambert、Nathan 等人。"Tülu 3：推动开源语言模型后训练的前沿"。发表于（2024年）。Li、Haonan 等人。《CMMLU：在中国测量大规模多任务语言理解》。2024年。arXiv：2306.09212[cs.CL]。网址：https://arxiv.org/abs/2306.09212。Li、Xi-Lin。"预条件随机梯度下降"。发表于《IEEE 神经网络与学习系统汇刊》29.5（2018年5月），第 1454–1466 页。ISSN：2162-2388。DOI：10.1109/tnnls.2017.2672978。网址：http://dx.doi.org/10.1109/TNNLS.2017.2672978。—— 用于 SGD 的矩阵李群预条件器。2018年。arXiv：1809.10232 [stat.ML]。网址：https://arxiv.org/abs/1809.10232。—— 使用李群的随机海森拟合。2024年。arXiv：2402.11858 [stat.ML]。网址：https://arxiv.org/abs/2402.11858。—— SGD 的黑盒李群预条件器。2022 年。arXiv：2211.04422 [stat.ML]。网址：https://arxiv.org/abs/2211.04422。Liu、Hong 等人。"Sophia：一种可扩展的随机二阶优化器，用于语言模型预训练"。发表于第十二届国际学习表示会议。2024年。网址：https://openreview.net/forum?id{xHDeA8Noi}。Loshchilov、Ilya 和 Frank Hutter。"解耦权重衰减正则化"。发表于国际学习表示会议。2019年。网址：https://openreview.net/forum?id{Bkg6RiCqY7}。

OLMo, Team et al. “2 OLMo 2 Furious”. In: *arXiv preprint arXiv:2501.00656* (2024).
OpenAI et al. *GPT-4 Technical Report*. 2024. arXiv: 2303.08774 [cs.CL]. URL: https://arxiv.org/abs/2303.08774.
Pethick, Thomas et al. *Training Deep Learning Models with Norm-Constrained LMOs*. 2025. arXiv: 2502.07529 [cs.LG]. URL: https://arxiv.org/abs/2502.07529.
Pooladzandi, Omead and Xi-Lin Li. *Curvature-Informed SGD via General Purpose Lie-Group Preconditioners*. 2024. arXiv: 2402.04553 [cs.LG]. URL: https://arxiv.org/abs/2402.04553.
Rajbhandari, Samyam et al. “ZeRO: Memory optimizations Toward Training Trillion Parameter Models”. In: (Nov. 2020), pp. 1–16. DOI: 10.1109/sc41405.2020.00024. URL: http://dx.doi.org/10.1109/SC41405.2020.00024.
Roy, Olivier and Martin Vetterli. “The effective rank: A measure of effective dimensionality”. In: *2007 15th European Signal Processing Conference*. 2007, pp. 606–610.
Shazeer, Noam. *Fast Transformer Decoding: One Write-Head is All You Need*. 2019. arXiv: 1911.02150 [cs.NE]. URL: https://arxiv.org/abs/1911.02150.
Shoeybi, Mohammad et al. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. 2020. arXiv: 1909.08053 [cs.CL]. URL: https://arxiv.org/abs/1909.08053.
Suzgun, Mirac et al. *Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them*. 2022. arXiv: 2210.09261 [cs.CL]. URL: https://arxiv.org/abs/2210.09261.
Team, Gemini et al. *Gemini: A Family of Highly Capable Multimodal Models*. 2024. arXiv: 2312.11805 [cs.CL]. URL: https://arxiv.org/abs/2312.11805.
Team, Gemma et al. “Gemma 2: Improving open language models at a practical size”. In: *arXiv preprint arXiv:2408.00118* (2024).
Team, Kimi. “Kimi k1.5: Scaling Reinforcement Learning with LLMs”. In: (2025).
Vyas, Nikhil et al. “SOAP: Improving and Stabilizing Shampoo using Adam”. In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: https://openreview.net/forum?id=IDxZhXrpNf.
Wang, Yubo et al. *MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark*. 2024. arXiv: 2406.01574 [cs.CL]. URL: https://arxiv.org/abs/2406.01574.
Wei, Tianwen et al. *CMATH: Can Your Language Model Pass Chinese Elementary School Math Test?* 2023. arXiv: 2306.16636 [cs.CL]. URL: https://arxiv.org/abs/2306.16636.
Yang, An et al. “Qwen2.5 Technical Report”. In: *arXiv preprint arXiv:2412.15115* (2024).
You, Jiacheng. *Jiacheng You’s discussion on Muon’s Update RMS*. 2025. URL: https://x.com/YouJiacheng/status/1890094769386451309.
Yuan, Huizhuo et al. *MARS: Unleashing the Power of Variance Reduction for Training Large Models*. 2024. arXiv: 2411.10438 [cs.LG].

OLMo, 团队等. “2 OLMo 2 愤怒”. 载于: *arXiv* 预印本 *arXiv:2501.00656* (2024). OpenAI 等 *GPT-4* 技术报告. 2024. arXiv: 2303.08774 [cs.CL]. URL: https://arxiv.org/abs/2303. 08774. Pethick, Thomas 等 使用范数约束*LMO*训练深度学习模型. 2025. arXiv: 2502.07529[cs.LG]. URL: https://arxiv.org/abs/2502.07529. Pooladzandi, Omead 和 Xi-Lin Li. 通过通用李群预处理器实现曲率信息*SGD*. 2024. arXiv: 2402.04553 [cs.LG]. URL: https://arxiv.org/abs/2402.04553. Rajbhandari, Samyam 等. “ZeRO: 内存优化面向训练万亿参数模型”. 载于: (2020年11月), 第1–16页. DOI: 10.1109/sc41405.2020.00024. URL: http://dx.doi.org/10.1109/SC41405.2020. 00024. Roy, Olivier 和 Martin Vetterli. “有效秩: 一种有效维度的度量”. 载于: *2007* 第*15*届欧洲信号处理会议. 2007, 第606–610页. Shazeer, Noam. 快速*Transformer*解码*:* 一个写头就够了. 2019. arXiv: 1911.02150 [cs.NE]. URL: https://arxiv.org/abs/1911.02150. Shoeybi, Mohammad 等. *Megatron-LM:* 使用模型并行性训练十亿参数语言模型. 2020. arXiv: 1909.08053 [cs.CL]. URL: https://arxiv.org/abs/1909.08053. Suzgun, Mirac 等. 挑战*BIG-Bench*任务和思维链是否能够解决它们. 2022. arXiv: 2210.09261 [cs.CL]. URL: https://arxiv.org/abs/2210.09261. 团队, Gemini 等. *Gemini:* 一系列高度强大的多模态模型. 2024. arXiv: 2312.11805 [cs.CL]. URL: https://arxiv.org/abs/2312.11805. 团队, Gemma 等. “Gemma 2: 在实用规模上改进开放语言模型”. 载于: *arXiv* 预印本 *arXiv:2408.00118* (2024). 团队, Kimi. “Kimi k1.5: 使用LLM扩展强化学习”. 载于: (2025). Vyas, Nikhil 等. “SOAP: 改进和稳定洗发水使用Adam”. 载于: 第十三届学习表示国际会议. 2025. URL: https://openreview.net/forum?id=IDxZhXrpNf. Wang, Yubo 等. *MMLU-Pro:* 一个更稳健和具有挑战性的多任务语言理解基准. 2024. arXiv: 2406.01574 [cs.CL]. URL: https://arxiv.org/abs/2406.01574. Wei, Tianwen 等. *CMATH:* 您的语言模型能通过中文小学数学测试吗？ 2023. arXiv: 2306.16636 [cs.CL]. URL: https://arxiv.org/abs/2306.16636. Yang, An 等. “Qwen2.5 技术报告”. 载于:*arXiv* 预印本 *arXiv:2412.15115* (2024). You, Jiacheng. *Jiacheng You*对*Muon*更新*RMS*的讨论. 2025. URL: https://x.com/YouJiacheng/ status/1890094769386451309. Yuan, Huizhuo 等. *MARS:* 释放方差减少为训练大型模型的强大功能. 2024. arXiv: 2411.10438 [cs.LG].

A Update RMS

Proof of Lemma 1

Proof. Without loss of generality, consider the orthogonal matrices $U \in \mathbb{R}^{n\times n}$ and $V \in \mathbb{R}^{m\times m}$ where $n \geq m \geq r$. We will show that for $X = U_{[:,:r]}V_{[:r,:]}$ (the update of the Muon has the same format), the RMS value is $\sqrt{r/mn}$. From the definition of matrix multiplication:

$$X_{i,j} = \sum_{k=1}^{r} U_{i,k}V_{k,j}$$

The RMS can be expressed as:

$$\begin{aligned}
\text{RMS}(X)^2 &= \frac{1}{mn}\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{k=1}^{r} U_{i,k}^2 V_{k,j}^2 \\
&= \frac{1}{mn}\sum_{k=1}^{r}\left(\sum_{i=1}^{n} U_{i,k}^2\right)\left(\sum_{j=1}^{m} V_{k,j}^2\right) \\
&= \frac{1}{mn}\sum_{k=1}^{r} 1 \\
&= \frac{r}{mn}
\end{aligned}$$

Therefore, $\text{RMS}(X) = \sqrt{r/mn}$. For the common case where the matrices are full-rank, $r = m$, yielding $\text{RMS}(X) = \sqrt{1/n}$. □

Consistent Update RMS Across Muon and AdamW As discussed in 2.2, we'd like to match the update RMS between Muon and AdamW optimizers. This is validated by experiments on small-scale models. We set Muon's Update RMS in the range of $[0.05, 0.1, 0.2, 0.4, 0.8]$ and AdamW as baseline. We reported the loss and representative weight matrix RMS at 2k steps (about 2B tokens) in the Table 8. From the results, we find that 0.2 RMS and 0.4 RMS performed similarly and much better than other settings. These findings are consistent with our empirical observation that AdamW's update RMS is in the range of $0.2 \sim 0.4$. We opted to control the update RMS of Muon to 0.2.

Table 8: Muon Update RMS Experiments

Optimizer	AdamW	0.05 RMS*	0.1 RMS	0.2 RMS	0.4 RMS	0.8 RMS
LM training loss	3.512	3.355	3.239	**3.198**	3.199	3.386
LM validation loss	3.679	3.503	3.374	3.325	**3.314**	3.543
AttnQ weight RMS	1.01e-2	5.74e-3	8.44e-3	1.57e-2	2.95e-2	7.23e-2
Mlp weight RMS	1.25e-2	8.01e-3	1.27e-2	2.35e-2	4.51e-2	8.73e-2

*Except the first column, all other candidates are using Muon with controlled RMS.

B AdamW Baseline Scaling Law

To ensure the fairness and accuracy of our experiments, we conducted a series of experiments on our proprietary dataset to derive scaling law parameters that are optimal for AdamW. This includes determining the optimal model size(N), number of training tokens(D), learning rate(η), batch size(B) under a constrained computational budget (FLOPs, C). (Kaplan et al. 2020; Hoffmann et al. 2022; Bi et al. 2024) Table 9 presents the results of our systematic parameter search process.

Table 9: Empirical Relationships Between Scaling Law Parameters and Computational Budget (FLOPs)

$N(C)$	$D(C)$	$\eta(C)$	$B(C)$
$0.0483359 \cdot C^{0.5112684}$	$3.4480927 \cdot C^{0.4887316}$	$0.0127339 \cdot C^{-0.0574752}$	$0.0065202 \cdot C^{0.4137915}$

A 更新均方根

证明引理<样式
id='1'>1</样式>
证明。 不失一般性，考虑正交矩阵 $U \in \mathbb{R}^{n\times n}$ 和 $V \in \mathbb{R}^{m\times m}$，其中 $n \geq m \geq r$。我们将证明对于 $X = U_{[:,:r]}V_{[:r,:]}$（Muon 的更新具有相同格式），均方根值为 $\sqrt{r/mn}$。根据矩阵乘法定义：

$$X_{i,j} = \sum_{k=1}^{r} U_{i,k}V_{k,j}$$

均方根可以表示为：

$$\begin{aligned}
\text{RMS}(X)^2 &= \frac{1}{mn}\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{k=1}^{r} U_{i,k}^2 V_{k,j}^2 \\
&= \frac{1}{mn}\sum_{k=1}^{r}\left(\sum_{i=1}^{n} U_{i,k}^2\right)\left(\sum_{j=1}^{m} V_{k,j}^2\right) \\
&= \frac{1}{mn}\sum_{k=1}^{r} 1 \\
&= \frac{r}{mn}
\end{aligned}$$

因此，$\text{RMS}(X) = \sqrt{r/mn}$。对于矩阵满秩的常见情况，$r = m$，得到 $\text{RMS}(X) = \sqrt{1/n}$。 □

Muon 和 AdamW 的更新均方根一致性如 2.2 中所述，我们希望匹配 Muon 和 AdamW 优化器之间的更新均方根。这一点通过在小规模模型上的实验得到了验证。我们将 Muon 的更新均方根设置为 $[0.05, 0.1, 0.2, 0.4, 0.8]$ 范围内，并将 AdamW 作为基线。我们在表 8 中报告了 2k 步（约 2B 个 token ）时的损失和代表性权重矩阵均方根。从结果来看，0.2 均方根和 0.4 均方根表现相似，并且远优于其他设置。这些发现与我们的经验观察一致，即 AdamW 的更新均方根在 $0.2 \sim 0.4$ 范围内。因此，我们选择将 Muon 的更新均方根控制在 0.2。

表 8：Muon 更新均方根实验

优化器	AdamW	0.05 均方根*	0.1 均方根	0.2 均方根	0.4 均方根	0.8 均方根
LM训练损失	3.512	3.355	3.239	**3.198**	3.199	3.386
LM验证损失	3.679	3.503	3.374	3.325	**3.314**	3.543
AttnQ权重均方根	1.01e-2	5.74e-3	8.44e-3	1.57e-2	2.95e-2	7.23e-2
MLP权重RMS	1.25e-2	8.01e-3	1.27e-2	2.35e-2	4.51e-2	8.73e-2

*除第一列外，其他候选者均使用受控均方根的Muon。

B AdamW基线缩放定律

为了确保我们实验的公平性和准确性，我们在专有数据集上开展了一系列实验，以推导出对 AdamW 最优的缩放规律参数。这包括在受限的计算预算（FLOPs,C）下确定最佳模型大小(N)、训练令牌数量(D)、学习率(η)和批大小(B)。(Kaplan et al. 2020; Hoffmann et al. 2022; Bi et al. 2024) 表 9 展示了我们系统参数搜索过程的结果。

表9：缩放定律参数与计算预算（FLOPs）之间的经验关系

$N(C)$	$D(C)$	$\eta(C)$	$B(C)$
$0.0483359 \cdot C^{0.5112684}$	$3.4480927 \cdot C^{0.4887316}$	$0.0127339 \cdot C^{-0.0574752}$	$0.0065202 \cdot C^{0.4137915}$

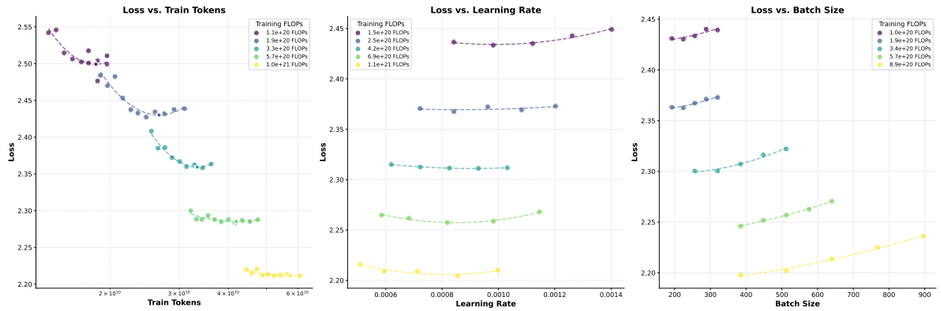


Figure 5: Optimization Landscapes for Scaling Law Hyper-parameters Across FLOPs Budgets

Hyper-Parameters Search To systematically identify optimal scaling law hyper-parameters in the AdamW baseline, we adopted a multistage search protocol. First, we selected multiple computational budgets (FLOPs levels) and initialized model sizes, learning rates, and batch sizes based on empirical guidelines from prior studies. For each fixed FLOPs constraint, we varied the model size N while adjusting the training token count D inversely to maintain $C = 6ND$, thereby exploring the trade-off between model capacity and data efficiency. Each configuration was trained to convergence, and the validation loss was recorded to determine the Pareto-optimal combinations of N and D. Subsequently, with the optimal $N - D$ pairs fixed, we refined the learning rate and batch size through grid searches, ensuring stability and convergence across configurations. To mitigate local minima and enhance robustness, this iterative procedure was repeated 2–3 times, progressively narrowing the hyper-parameter space.

The optimization process is further illustrated in Figure 5, which depicts the loss landscapes as functions of training tokens, learning rate, and batch size across varying FLOPs budgets. Each bowl-shaped curve represents the loss surface for a specific FLOPs level, with a distinct global minimum corresponding to the optimal hyper-parameter configuration.

C Model Architecture

Muon is agnostic to model architectures, and we used a model similar to Deepseek-V3-Small as described in DeepSeek-AI et al. 2024, because it is a strong model with open weights as a baseline. We made several small modifications in the Moonlight model and listed them here:

Multi-token Prediction (MTP) MTP has not shown significant benefits to pretraining in our experiments. For simplicity, we do not introduce MTP layers into the Moonlight model.

Auxfree Bias Update In DeepSeek-AI et al. 2024, auxfree bias is updated by: $b_i = b_i + u \times \text{sign}(e_i)$, where u is the update ratio, b_i is the bias for the ith expert, and e_i is the expert's violating ratio. We slightly modified the update rule as: $b_i = b_i + u \times (\text{sign}(e_i) - \text{sign}(e).\text{mean}())$, where $\text{sign}(e).\text{mean}()$ is the average of the signs of all expert's violating ratio, in order to control the magnitude of the bias, while does not change the topk selection logic.

Gate Scaling Factor Deepseek-V2-Lite did not use the gate scaling factor, and Deepseek-V3 used a scaling factor of 2.5. We used a scaling factor of 2.446 to control a similar output rms like dense models. The code for calculating our gate scaling factor can be found in Figure 6.

D Training Stability

No Loss or Grad Norm Spike The Moonlight training process was very smooth and we did not meet any loss spike or gradient norm spike. The loss and grad norm curve can be seen in Figure 7 (Moonlight is colored in blue and Moonlight-A trained by AdamW is colored in red)

Max Attention Logit During training, we observed that while both the training loss and gradient norm remained stable throughout the process, the maximum attention logit (computed as the single largest logit value across the global

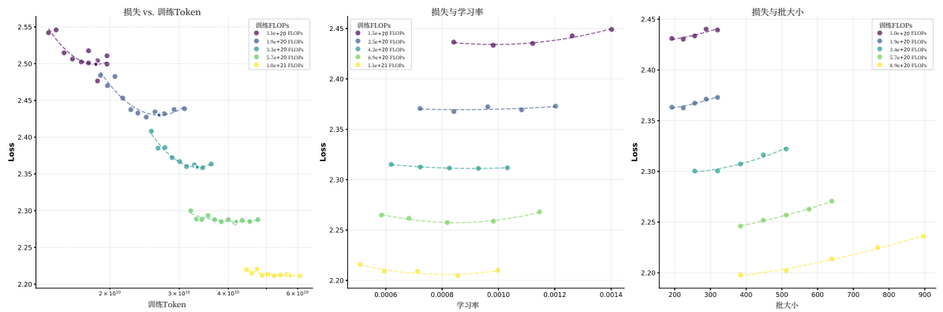


图 5：不同 FLOPs 预算下的缩放律超参数优化景观

超参数搜索 为了系统地识别AdamW基线中的最优缩放律超参数，我们采用了多阶段搜索协议。首先，我们选择了多个计算预算（FLOPs级别）和初始化模型大小、学习率和批大小，这些基于先前研究中的经验指南。对于每个固定的FLOPs约束，我们改变模型大小 N，同时反向调整训练令牌计数 D 以保持$C = 6ND$，从而探索模型容量和数据效率之间的权衡。每个配置都训练至收敛，并记录验证损失以确定 N 和 D的Pareto最优组合。随后，在最优的 $N - D$ 对固定后，我们通过网格搜索细化学习率和批大小，确保配置间的稳定性和收敛性。为了减轻局部最小值并增强鲁棒性，该迭代过程重复了2-3次，逐步缩小超参数空间。

优化过程在图 5 中进一步说明，该图描绘了训练令牌、学习率和批大小的损失景观，这些参数随不同的 FLOPs 预算变化。每条碗状曲线代表特定 FLOPs 级别的损失表面，对应一个独特的全局最小值，该最小值与最优超参数配置相对应。

C 模型架构

Muon 对模型架构具有兼容性，我们使用了与 Deepseek-V3-Small 类似的模型，如 DeepSeek-AI 等人 2024 年所述，因为它是一个具有开放权重的强模型，可作为基线。我们在 Moonlight 模型中进行了几项小修改，并在此列出：

多token预测 **n (MTP)** MTP在我们的实验中尚未显示出对预训练的显著优势。F
为简化起见，我们不在Moonlight模型中引入MTP层。

Auxfree Bias Update 在DeepSeek-AI等2024年的研究中，auxfree偏差通过 $b_i = b_i + u \times \text{sign}(e_i)$进行更新，其中 u 是更新比率，b_i 是第i个专家的偏差，而 e_i 是专家违反率。我们略微修改了更新规则为：$b_i = b_i + u \times(\text{sign}(e_i) - \text{sign}(e).\text{mean}())$，其中$\text{sign}(e).\text{mean}()$是所有专家违反率的符号平均值，以控制偏差的幅度，同时不改变topk选择逻辑。

门缩放因子 Deepseek-V2-Lite没有使用门缩放因子，而Deepseek-V3使用了2.5的缩放因子。我们使用了2.446的缩放因子来控制密集模型相似的输出RMS。计算我们门缩放因子的代码可以在图6中找到。

D 训练稳定性

无损失或梯度范数尖峰 Moonlight 训练过程非常平滑，我们没有遇到任何损失尖峰或梯度范数尖峰。损失和梯度范数曲线可以在图 7 中看到（Moonlight 以蓝色显示，由 AdamW 训练的 Moonlight-A 以红色显示）

最大注意力logit 在训练过程中，我们观察到训练损失和梯度范数在整个过程中都保持稳定，但最大注意力logit（计算为全局范围内单个最大的logit值）在初始训练阶段于特定层表现出明显的上升趋势，超过了100的阈值。值得注意的是，与替代优化器相比，AdamW在控制这一指标方面表现更健康。

```python
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def calc_gate_scaling_factor(num_experts: int, topk: int, iter_times: int):
    """Calculate the gate scaling factor for MoE.

    Args:
        num_experts (int): The number of experts.
        topk (int): The number of experts to select.
        iter_timers (int): The number of iterations.

    Returns:
        float: The gate scaling factor.
    """
    factors = []
    for _ in range(iter_times):

        # mock gaussian logits
        logits = np.random.randn(num_experts)
        # select topk logits
        p = np.sort(sigmoid(logits))[::-1]
        p = p[:topk]
        # renormalize
        p = p / p.sum()
        # calculate the scaling factor
        factors.append( 1/ (p**2).sum()**0.5)
    return np.mean(factors)
```

Figure 6: Python implementation for calculating the gate scaling factor.

batch) exhibited a distinct upward trajectory in specific layers during the initial training phase, exceeding a threshold of 100. Notably, AdamW demonstrated healthier behavior in controlling this metric compared to alternative optimizers.

To further investigate the impacts of this phenomenon, we introduced the large attention logits ratio metric, defined as the proportion of attention logits exceeding 100 within a batch. As shown in Fig.7, this ratio remained consistently low (about 10^{-4}), indicating that extreme large logit values were sparse. Furthermore, the maximum logit values gradually decrease as training progressed, suggesting that the optimization dynamics become healthier.

RMSNorm Gamma Weight Decay It is noteworthy that applying weight decay to the RMSNorm gamma parameter is crucial for ensuring training stability, as it effectively prevents excessively high output RMS values in each layer.

E Comparison with More Expensive Models

Table 10 presents a comparative analysis between our Moonlight model (optimized with Muon) and publicly available models trained with greater computational resources, including `LLama3.1-8B` (Grattafiori et al. 2024), `Gemma-9B` (Gemma Team et al. 2024) and `Qwen2.5-7B` (Yang et al. 2024). Figure 8 illustrates the GSM8k performance benchmarks of Moonlight against comparable models in the field.

F Singular Value Distributions of Weight Matrices

We visualize the singular value distributions of weight matrices by plotting a line graph of its singular values in descending order for each matrix, normalized by the largest one. As shown in Figures 9 and 10, we find that, for most of the weight matrices, the singular value distributions of them optimized by Muon are more flattened than that of AdamW, which further confirms the hypothesis that Muon can provide a more diverse spectrum of updates.


```python
import numpy as np

def sigmoid (x ):
    return    1 / (1  +     np . exp (-x ))

def calc_门_缩放_因子 ( num_专家 : int ,                    topk:  int ,  迭代_次数 : int ):
    """ 计算门缩放因子                          用于 MoE .

    参数:
        num_experts (int): 专家的数量。
        topk (int): 选择的专家数量。          topk (int): 选择的专家数量。
        iter_计时器     timers (int): 迭代次数。

    返回:
        float:          门缩放因子          。
    """
    因子       = []
    for  _  在范围内(迭代_次):

        # 模拟     高斯logits
        logits   =np . random . randn ( num_experts )
        # 选择topk logits
        p  =  np . sort ( sigmoid ( logits )) [:: -1]
        p  =  p [: topk ]
        #  renormalize
        p  =  p  /  p。sum()
        # 计算扩展                                  因子
        因素。append(1/)          (p**2) . sum ()**0.5)
    返回 np.mean(factors)
```

图6：计算门缩放因子的Python实现。

在初始训练阶段于特定层表现出明显的上升趋势，超过了100的阈值。值得注意的是，AdamW在控制这一指标方面表现更健康。

为了进一步研究这种现象的影响，我们引入了大注意力logit比率指标，定义为一批次中超过100的注意力logit的比例。如图7所示，该比率始终保持较低（约 10^{-4}），表明极端大的logit值较为稀疏。此外，随着训练的进行，最大logit值逐渐减小，这表明优化动态变得更加健康。

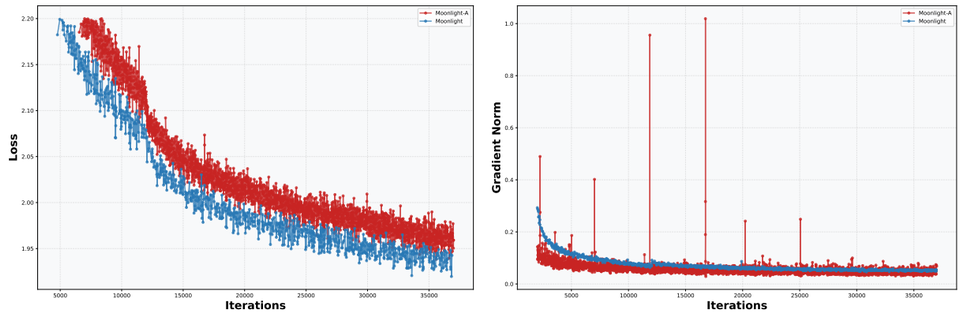
RMSNorm Gamma权重衰减 值得注意的是，将权重衰减应用于RMSNorm gamma参数对于确保训练稳定性至关重要，因为它有效地防止了每层中过高的输出RMS值。

E 与更昂贵的模型比较

表10展示了我们的Moonlight模型（使用Muon优化）与使用更多计算资源训练的公开可用模型之间的比较分析，包括LLama3.1-8B（Grattafiori等人 2024）、Gemma-9B（Gemma团队等人 2024）和Qwen2.5-7B（Yang等人 2024）。图8展示了Moonlight与该领域可比模型在GSM8k性能基准上的表现。

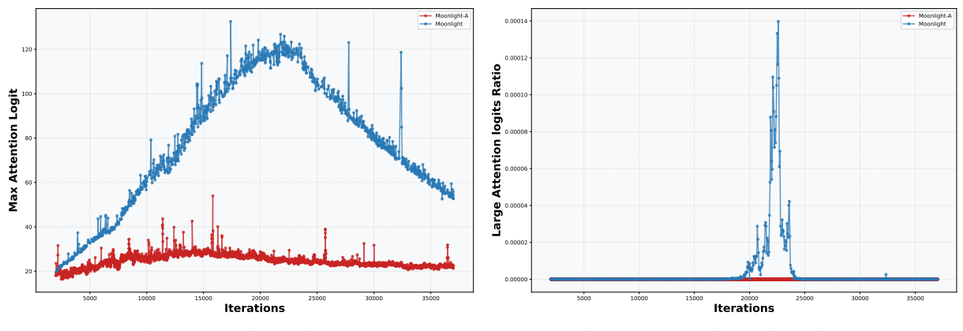
F 奇异值分布(权重矩阵)

我们通过绘制每个矩阵按奇异值降序排列的线图来可视化权重矩阵的奇异值分布，并将其归一化到最大值。如图9和图10所示，我们发现，对于大多数权重矩阵，由Muon优化的奇异值分布比AdamW的更平坦，这进一步证实了Muon可以提供更多样化的更新谱的假设。



(a) Training Loss

(b) Gradient Norm



(c) Max Attention Logit (Layer 1)

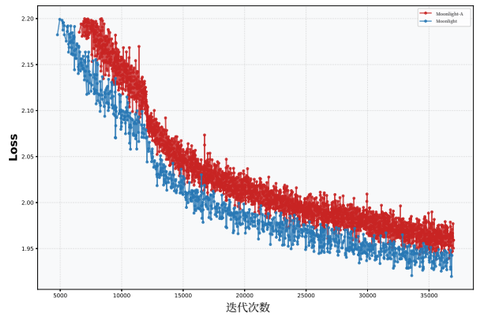
(d) Large Attention Logits Ratio (Layer 1)

Figure 7: Training dynamics comparison between Moonlight and Moonlight-A

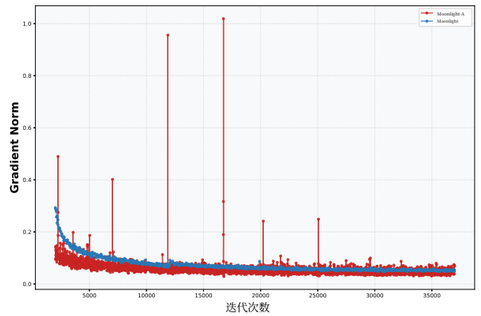
Table 10: Comparison of different models on various benchmarks.

	Benchmark (Metric)	**Moonlight**	**LLAMA3.1-8B**	**Gemma2-9B**	**Qwen2.5-7B**
			Larger Training Compute Model		
	Activated Param†	2.24B	7.38B	8.32B	6.83B
	Total Params†	15.29B	7.38B	8.32B	6.83B
	Training Tokens	5.7T	15T	8T	18T
	Optimizer	Muon	AdamW	Unknown	Unknown
English	MMLU	70.0	66.7	71.3	74.2
	MMLU-pro	42.4	37.1	44.7	45.0
	BBH	65.2	57.7	68.2	70.4
	TriviaQA‡	66.3	70.3	-	60.0
Code	HumanEval	48.1	37.2	37.8	57.9
	MBPP	63.8	47.6	62.2	74.9
Math	GSM8K	77.4	57.2	70.7	85.4
	MATH	45.3	20.3	37.7	49.8

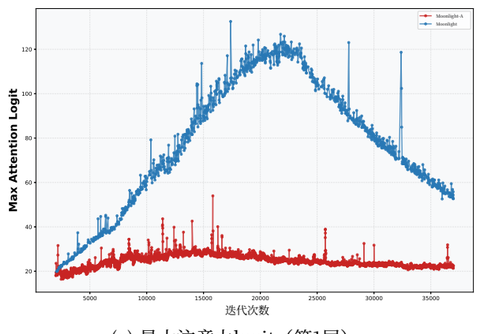
† The reported parameter counts exclude the embedding parameters.‡ We test all listed models with the full set of TriviaQA.



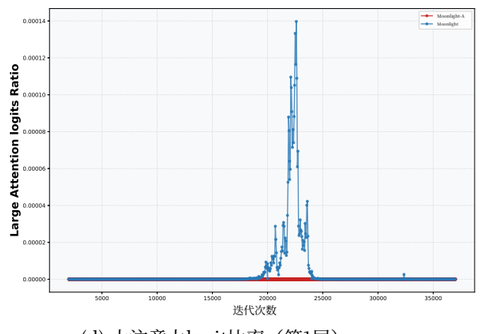
(a) 训练损失



(b) 梯度范数



(c) 最大注意力logit（第1层）



(d) 大注意力logit比率（第1层）

图7：Moonlight与Moonlight-A的培训动态对比

表10：不同模型在各种基准测试上的比较。

	基准（指标）	**Moonlight**	**LLama3.1-8B**	**Gemma2-9B**	**Qwen2.5-7B**
			更大训练计算模型		
	激活参数†	2.24B	7.38B	8.32B	6.83B
	总参数†	15.29B	7.38B	8.32B	6.83B
	训练令牌	5.7T	15T	8T	18T
	优化器	Muon	AdamW	未知	未知
英语	MMLU	70.0	66.7	71.3	74.2
	MMLU-pro	42.4	37.1	44.7	45.0
	BBH	65.2	57.7	68.2	70.4
	TriviaQA‡	66.3	70.3	-	60.0
Code	HumanEval	48.1	37.2	37.8	57.9
	MBPP	63.8	47.6	62.2	74.9
Math	GSM8K	77.4	57.2	70.7	85.4
	MATH	45.3	20.3	37.7	49.8

† 报告的参数数量包括 嵌入参数。‡ 我们测试了所有列出的模型 ，并使用完整的TriviaQA参数集进行测试。

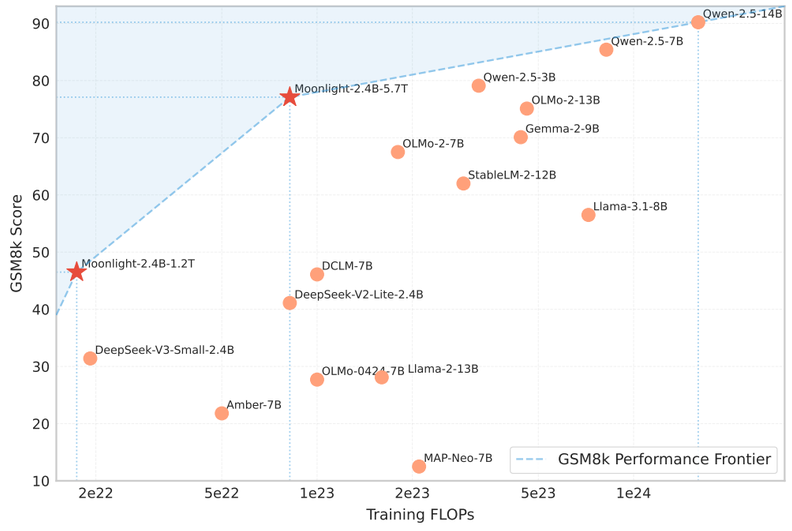


Figure 8: The GSM8k performance of our Moonlight model optimized with Muon and other comparable models.

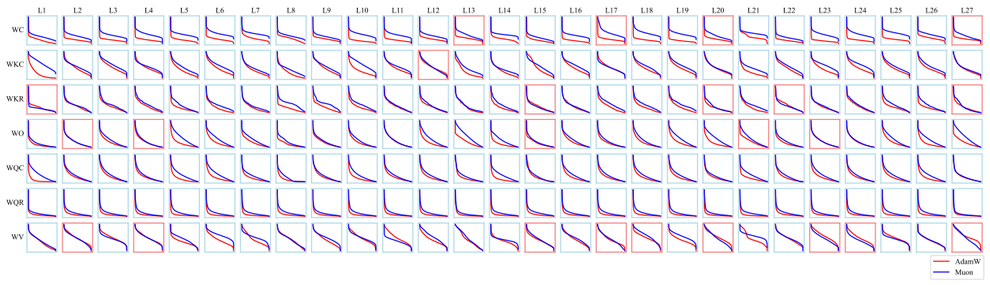


Figure 9: Distribution of singular values for each weight matrix in the attention layers. We use WC to denote the weight matrices at each layer that compress the hidden states to the shared latent spaces for keys and values, WV to denote the weight matrices up-projecting the values from the latent space, WO to denote the output projection matrices, and WKR, WKC, WQR and WQC to denote the projection matrices for the part of keys and queries with and without RoPE respectively. We set the spines of each line graph red if the corresponding weight matrix optimized by Muon has a lower singular entropy than AdamW.

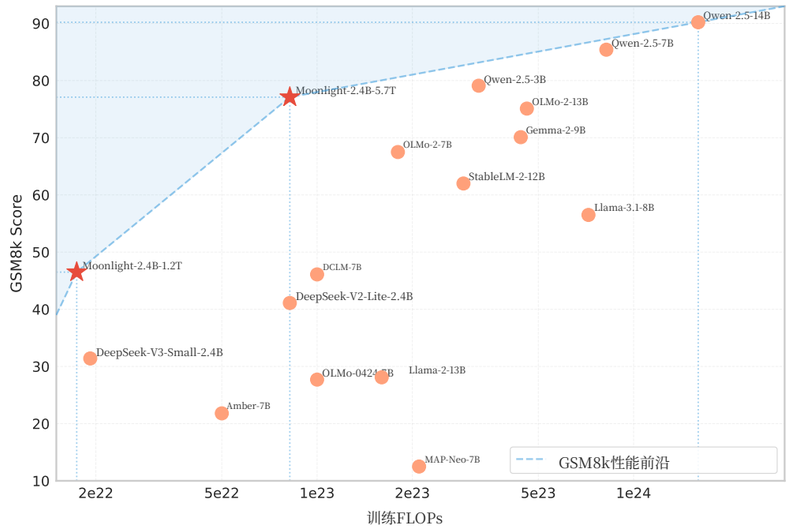


图8：GS 我们的Moonlight模型使用Muon和其他可比较模型优化的M8k性能 ls.

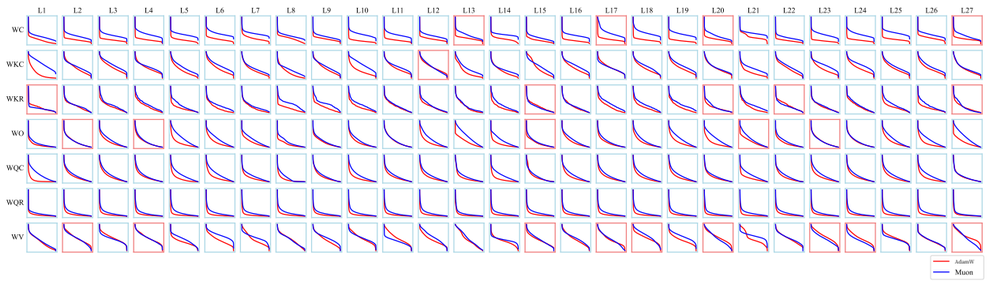


图9：注意力层中每个权重矩阵的奇异值分布。我们用WC表示在每一层压缩隐藏状态到键和值共享潜在空间的权重矩阵，用WV表示从潜在空间上投影值的权重矩阵，用WO表示输出投影矩阵，以及用WKR、WKC、WQR和WQC分别表示带有和没有RoPE的键和查询部分的投影矩阵。如果由Muon优化的对应权重矩阵的奇异熵低于AdamW，则将每条线图的边框设置为红色。

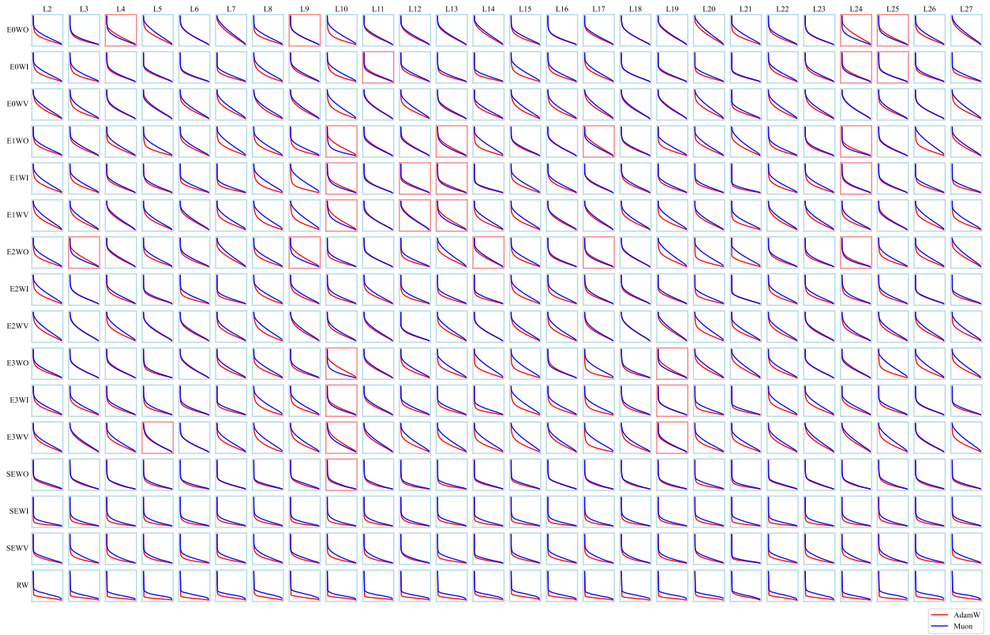


Figure 10: Distribution of singular values for each weight matrix in the feed-forward network (FFN) layers. We use WI, WV and WO to denote the weight matrices involved in the FFN layer with SwiGLU activation function, where WI represents the input projection to the Swish_1 function, WV represents the extra input projection interacting with Swish_1 activations, and WO represents the output projection. We use E0, E2, E3 to denote three arbitrarily selected expert models and SE to denote the weights in the shared expert model. We use RW to denote the weights in the router. We set the spines of each line graph red if the corresponding weight matrix optimized by Muon has a lower singular entropy than AdamW.

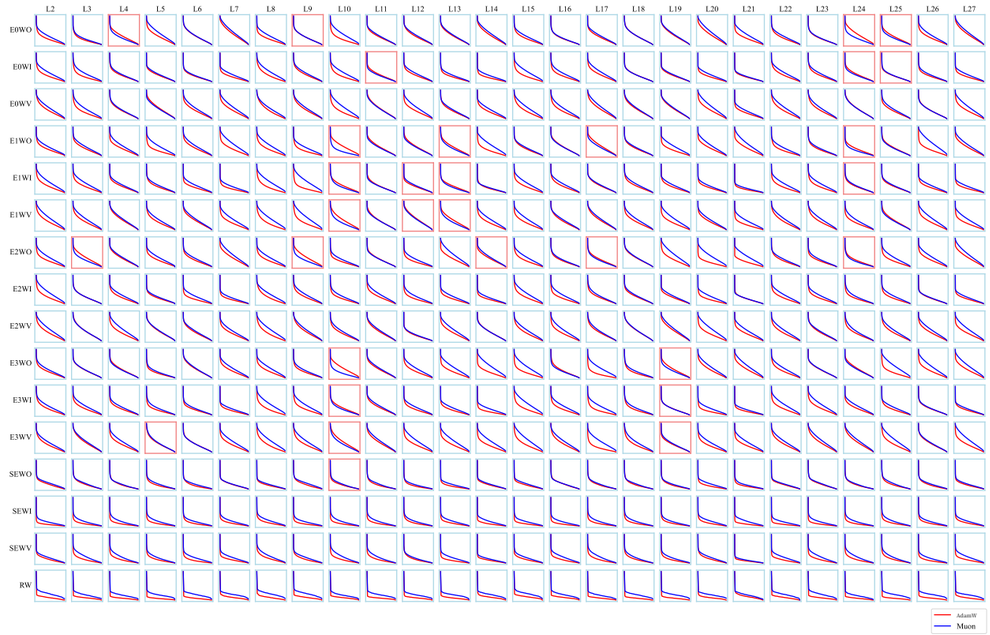


图 10：前馈网络（FFN）层中每个权重矩阵的奇异值分布。我们使用 WI、WV 和 WO 来表示涉及 SwiGLU 激活函数的 FFN 层的权重矩阵，其中 WI 表示到 Swish_1 函数的输入投影，WV 表示与 Swish_1 激活交互的额外输入投影，WO 表示输出投影。我们使用 E0、E2、E3 表示三个任意选择的专家模型，并使用 SE 表示共享专家模型中的权重。我们使用 RW 表示路由器中的权重。如果 Muon 优化的相应权重矩阵的奇异熵低于 AdamW，则将每条线图的脊柱设置为红色。